

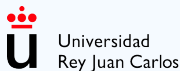
Big Data Processing II

Tema 1: Procesamiento de Big Data en Entornos Distribuidos

Alberto Fernández Isabel
Natalia Madrueño Sierro
Rubén Rodríguez Fernández

DSLAB

Febrero 2026





Material docente en abierto de la Universidad Rey Juan Carlos

Asignatura: Big Data Processing II

Titulación: Máster Universitario en Análisis de Datos Deportivos

Autores: Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández

Fecha: 2026

© 2026 Equipo docente. Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional.

Los logotipos, marcas y referencias de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

Depósito institucional: <https://burjcdigital.urjc.es>



- Comprender la transición del procesamiento **batch** a **streaming** en analítica deportiva
- Identificar las características de los sistemas de **mensajería distribuida**
- Conocer los fundamentos de los motores de **procesamiento en tiempo real**
- Familiarizarse con los **principios de diseño** de arquitecturas distribuidas
- Relacionar estos conceptos para casos de uso en el **análisis deportivo en vivo**



- 1 **Introducción**
- 2 **Mensajería distribuida**
- 3 **Procesamiento en streaming**
- 4 **Diseño a nivel de sistema**



- 1 **Introducción**
- 2 Mensajería distribuida
- 3 Procesamiento en streaming
- 4 Diseño a nivel de sistema



- **El análisis de datos puede apoyar la toma de decisiones durante el evento**
 - Antes $\Rightarrow$ análisis después del entrenamiento o partido
 - Ahora $\Rightarrow$ decisiones en vivo sobre táctica, rotaciones y gestión de la carga
- El foco se desplaza de **datos estáticos** a **eventos continuos** procesados en tiempo real



- **El valor del dato deportivo cae con el tiempo**

- Información procesada tarde es una oportunidad perdida
- Lo correcto a destiempo puede no ser de utilidad en la práctica

- **Ejemplos de analítica deportiva en vivo**

- Fatiga y lesión, con alertas durante el esfuerzo, no después
- Tracking de jugadores, detectando patrones de presión, desmarques y desajustes en vivo



- **Procesamiento de datos en batch**

- Se recopila un conjunto de datos y se analiza después
- Se asume que el dataset está completo antes de procesar

- **Casos de uso en el ámbito deportivo**

- Reporting / scouting, histórico, modelos offline y análisis exploratorio
- Presenta una alta latencia, el resultado llega cuando la acción ya terminó



- **Se usa un conjunto de datos cerrado y finito**
 - Hay un final conocido, se puede procesar todo
 - *input cerrado* → procesamiento → *output final*
- **Optimiza la capacidad global del sistema**
 - Throughput → cuántos datos se procesan por unidad de tiempo
- **No prioriza el tiempo de respuesta individual**
 - Latencia → cuánto tarda un evento en producir un resultado



Procesamiento por lotes

Esquema conceptual original de la edición abierta

Fuente: <https://www.decube.io/post/streaming-vs-batch-data>



- **Tecnologías habituales**

- Hadoop y MapReduce para grandes lotes distribuidos
- Spark batch como API de alto nivel e iteración más rápida

- **Almacenamiento**

- Data Lakes y Data Ware Houses para almacenar históricos

Ecosistema Hadoop

Esquema conceptual original de la edición abierta

Procesamiento distribuido con Spark

Esquema conceptual original de la edición abierta



- **Big Data se centra en volumen**

- Almacena y procesa grandes volúmenes de datos
- Procesamiento en batch analiza datos finitos
- Ejemplos incluyen datos históricos deportivos

- **Fast Data se centra en velocidad**

- Procesa y toma decisiones sobre los datos mientras se generan
- Procesamiento en streaming analiza flujos continuos
- Ejemplos incluyen datos de tracking en vivo



- **Datos finitos (bounded)**

- Datos cerrados con fin definido
- Se da un único resultado al final
- Caso particular de datos continuos

- **Datos continuos (unbounded)**

- Flujo de datos potencialmente infinito
- Hay que producir resultados en continuo



- **La temperatura del dato se refiere a su frescura y utilidad**
- **Datos fríos (cold)**
 - Histórico, auditoría y entrenamiento de modelos
 - Procesamiento en batch opera sobre frío
- **Datos calientes (hot)**
 - Máximo valor ahora, requieren decisión inmediata
 - Procesamiento en streaming opera sobre caliente



- **Concepto de panta rei o “todo fluye”**
 - La realidad está en cambio continuo
 - Es una secuencia de eventos con contexto
 - No se piensa en una tabla final, sino en una historia que se actualiza
- **Cambio de mentalidad hacia flujos dinámicos de eventos**



- **Computación incremental sobre eventos**
 - Se prioriza la velocidad en la toma de decisiones
 - Procesa eventos conforme llegan, sin esperar a un dataset completo
 - Genera resultados incrementales que se refinan con cada nuevo evento
- **Requiere un trade-off entre latencia, coste y exactitud**



Procesamiento continuo de eventos

Esquema conceptual original de la edición abierta

<https://www.decube.io/post/streaming-vs-batch-data>



- **Desacopla quién produce de quién consume**
 - Fuente → distribuidor → procesador → destino
- **Convierte eventos en señales útiles**
 - Métricas, alertas y dashboards
 - Cálculo en vivo de velocidad, fatiga y patrones tácticos



- **Se requiere un buffer intermedio que gestione el flujo continuo**
 - Amortigua picos de eventos que no llegan a ritmo constante
 - Desacopla productores y consumidores para que evolucionen independientemente
 - Retiene temporalmente eventos si el procesador se satura
 - Facilita reintentos ante fallos y, a veces, replay
- **Requiere una infraestructura de mensajería distribuida**



- **Se requiere procesar el flujo continuo de datos**
 - Transformar eventos continuos en señales útiles
 - Se puede hacer tratando cada evento de forma independiente
 - Se puede recordar eventos pasados para computar vistas acumuladas
- **Requiere un motor de procesamiento en streaming**



- **Piezas típicas de un sistema de streaming**

- Fuentes → sensores, wearables, logs
- Mensajería → Kafka, Pulsar, RabbitMQ
- Procesador → Flink, Spark Streaming, Storm, Beam
- Destinos → bases de datos, dashboards, data lake, alertas

- **Streaming no es una herramienta, es un sistema de piezas coordinadas**



Arquitectura distribuida de referencia

Esquema conceptual original de la edición abierta

<https://www.scaler.com/topics/spark-streaming-with-kafka/>



- 1 Introducción
- 2 Mensajería distribuida**
- 3 Procesamiento en streaming
- 4 Diseño a nivel de sistema



- **Los eventos no se producen a un ritmo constante**
 - Sin una infraestructura intermedia el sistema colapsa
 - Existen picos de carga y consumidores lentos
 - Las fuentes de datos emiten a ritmo irregular
 - Los consumidores no pueden ir sincronizados
- **Middleware Orientado a Mensajes (MOM)**
 - Infraestructura intermedia que absorbe picos y reparta trabajo



- **Integración punto a punto (P2P)**

- Muchas conexiones directas suponen un acoplamiento fuerte
- Cambios en contratos y formato implican integraciones frágiles
- Cada nuevo consumidor depende excesivamente de los productores

- **Arquitectura dirigida por eventos (EDA)**

- Productor → Distribuidor → Consumidor
- Productores y consumidores se encuentran desacoplados
- Añadir consumidores no implica cambiar productores

Arquitectura orientada a eventos y agentes

Esquema conceptual original de la edición abierta

<https://solace.com/blog/agentai-the-new-microservices/>



- **Infraestructura de eventos para datos en movimiento**
 - Ingiere, retiene y distribuye eventos de forma desacoplada
 - Absorbe picos mediante buffering y controles de carga
 - Fuentes → Analítica en vivo → Histórico
- **Sistemas de mensajería distribuida para eventos**



- **Un mensaje es el “sobre” que transporta un evento**
 - Unidad de comunicación → payload + headers
 - Foco en entrega → ack, routing, reintentos y colas
- **Un evento tiene un significado en el dominio**
 - Algo ocurrió (quién, qué, cuándo) + contexto
 - Foco en significado → métricas, alertas y reconstrucción de historia
- **Se transportan eventos dentro del payload de un mensaje**



- **Cola de mensajes (Queue)**

- Un evento lo procesa un único consumidor
- Útil para tareas y reparto de carga

- **Publicación/Suscripción (Pub/Sub)**

- Un evento lo reciben muchos consumidores (fan-out)
- Útil para métricas + alertas + persistencia en paralelo

Colas y publicación-suscripción

Esquema conceptual original de la edición abierta

<https://docs.cloud.google.com/solutions/event-driven-architecture-pubsub>



- **Los mensajes no siempre llegan en orden ni en tiempo**
 - Latencia en red, buffering y dispositivos que se desconectan y vuelven
 - Algunas métricas son provisionales y se corrigen si llega un evento tarde
- **El sistema de mensajería debe tolerar desorden y retardos**
- **Se debe decidir entre perder eventos o tolerar duplicados**



- **At-most-once**

- Más rápido $\Rightarrow$ mínima latencia y overhead
- Menos fiable $\Rightarrow$ puede perder mensajes

- **At-least-once**

- Más fiable $\Rightarrow$ entrega una o más veces
- Más lento $\Rightarrow$ más overhead/latencia

- **Exactly-once**

- Mismo resultado final (efecto único)
- No significa “cero duplicados en la red”
- Suele ser más lento y depende de la plataforma



- **Todo sistema de mensajería gestiona cuatro aspectos clave**
 - Canales para organizar mensajes en colas, topics o streams
 - Escalado y paralelismo mediante particiones o consumidores
 - Fiabilidad mediante acks, reintentos y persistencia
 - Consumo destructivo tras ack o replay histórico
- **Cada sistema los prioriza e implementa de forma distinta**



- **Brokers clásicos**

- Optimiza la entrega por mensaje
- *Smart broker* → más lógica y decisiones en el broker

- **Events logs**

- Optimiza retención, offsets y replay del flujo de eventos
- *Dumb broker, smart consumer* → más lógica y decisiones en consumidores

- **Tareas y routing vs histórico, fan-out y reproceso**



- **Optimiza entrega y routing por mensaje**
 - Colas y pub/sub con routing flexible (topics/keys/exchanges)
 - Patrones de integración habituales (filtering, routing, request/reply)
- **Poseen una alta fiabilidad operativa**
 - Acks y reintentos, control de redelivery
 - Dead-letter queues y políticas de error
- **Ideal para mensajes/tareas asíncronas e integración empresarial**
 - Ejemplos incluyen RabbitMQ, ActiveMQ, Amazon SQS, ...

Broker RabbitMQ

Esquema conceptual original de la edición abierta

Servicio de colas gestionado

Esquema conceptual original de la edición abierta



- **Optimiza el flujo de eventos como un log distribuido**
 - Append-only (commit log) con retención (por tiempo/tamaño)
 - Lectura por offsets → cada consumidor controla su progreso
- **Habilita fan-out, retención y replay**
 - Muchos consumidores independientes sobre el mismo flujo
 - Retención para recomputar vistas, auditoría y nuevos consumidores
- **Ideal para analítica en vivo y pipelines de datos**
 - Ejemplos incluyen Kafka, Pulsar, AWS Kinesis, ...

Plataforma de eventos Kafka

Esquema conceptual
original de la
edición abierta



- 1 Introducción
- 2 Mensajería distribuida
- 3 Procesamiento en streaming**
- 4 Diseño a nivel de sistema



- **Streaming procesa flujos continuos de eventos**
 - No se trata solo de hacerlo rápido
 - Se computa sobre un flujo continuo que no termina
- **Batch es un caso particular de un stream finito**



- **Dimensiones clave en el procesamiento en streaming**
 - Tiempo → ¿Cuándo ocurrió el evento vs cuándo lo proceso?
 - Estado → ¿Necesito recordar el pasado para procesar el presente?
 - Correctitud → ¿Qué garantías doy ante fallos, retrasos y duplicados?
- **Cada sistema en streaming prioriza unas dimensiones sobre otras**
- **Existen diferentes enfoques de procesamiento según estas dimensiones**



- **Simple Event Processing (SEP)**

- Filtra, enruta y transforma evento a evento
- Ejemplos incluyen filtrar posiciones imposibles

- **Event Stream Processing (ESP)**

- Consultas continuas, como ventanas, agregaciones y métricas online
- Ejemplos incluyen sprints por jugador en 5 min

- **Complex Event Processing (CEP)**

- Patrones complejos y temporales, como secuencias, correlación y anomalías
- Ejemplos incluyen análisis de presión sostenida



- **Procesamiento sin estado (stateless)**

- Cada evento se procesa de forma independiente
- Operaciones típicas incluyen map, filter y enrich

- **Procesamiento con estado (stateful)**

- El procesamiento depende de eventos previos
- Operaciones típicas incluyen agregaciones, joins, sesiones y ventanas

- **El estado introduce complejidad en el sistema**



- **Tiempo de proceso**

- Cuándo el sistema procesa el evento
- Es el reloj del sistema
- Monitoriza latencia y rendimiento

- **Tiempo de evento**

- Cuándo ocurrió en el mundo real
- Es el reloj del evento
- Necesario para métricas correctas

- **La verdad del juego es el tiempo de evento**

Tiempo de procesamiento y tiempo de evento

Esquema conceptual original de la edición abierta

Hesse, Guenter. (2022). A benchmark for enterprise stream processing architectures. 10.25932/publishup-56600.



- **Alinear el tiempo entre fuentes supone un reto**
 - Múltiples fuentes con relojes distintos
 - Desfases de reloj y necesidad de normalizar timestamps
 - Retrasos por red, buffering o dispositivos offline
 - Eventos llegan desordenados o en bloque tras reconexión
- **Ventanas en tiempo de evento con tolerancia a tardíos**
 - Métricas se calculan en tiempo de evento y no de proceso
 - El sistema acepta tardíos y aplica correcciones con triggers y watermarks



- **En flujos continuos no hay un fin definido**
 - Para agregar lo infinito se trocea en fragmentos finitos
 - Se agrupan eventos en ventanas en tiempo de evento
 - Cada ventana produce un resultado actualizable
- **El tipo de ventana depende de la pregunta**
 - Intervalos fijos, solapados o por inactividad (sesiones)
 - Es necesario definir tamaño, salto y tolerancia a tardíos
- **Mayor correctitud implica mayor complejidad**



- **Tumbling**

- Ventanas con segmentos contiguos de tamaño fijo y sin solape
- Cada evento cae en una única ventana

- **Hopping/Sliding**

- Ventanas fijas con un salto definido con posible solape
- Un evento puede contribuir a varias ventanas

- **Session (por inactividad)**

- Ventanas de tamaño variable delimitadas por un gap sin eventos
- Con tardíos, pueden requerir merge de sesiones

Ventanas de intervalo fijo

Esquema conceptual original de la edición abierta

Ventanas deslizantes

Esquema conceptual original de la edición abierta

Ventanas de sesión

Esquema conceptual original de la edición abierta



- **Triggers para emitir resultados por ventana**
 - Early → rápido y aproximado
 - On-time → cuando la ventana se considera completa
 - Late → actualizaciones por eventos tardíos
- **Streaming involucra resultados revisables**



- **En tiempo de evento los datos pueden llegar tarde y desordenados**
- **Un watermark indica el progreso del tiempo de evento**
 - El sistema necesita una noción de progreso para no esperar indefinidamente
 - Los eventos con $\text{timestamp} < \text{watermark}$ se consideran tardíos (late data)
 - A partir de ese instante asume que eventos más antiguos serán excepcionales
- **Decide cuándo cerrar ventanas y emitir resultados**
 - Agresivo $\rightarrow$ menor latencia, más tardíos y posibles correcciones
 - Conservador $\rightarrow$ más completitud/exactitud, mayor latencia y más estado



- **Se debe definir hasta cuándo se aceptan tardíos y correcciones**
 - Para eventos que llegan con $\text{timestamp} < \text{watermark}$
 - Se define un trade-off entre latencia, coste y exactitud
- **Políticas de late data más habituales**
 - Ignorar/descartar $\rightarrow$ menor coste pero pérdida de exactitud
 - Emitir correcciones $\rightarrow$ resultados actualizables pero mayor cómputo
 - Retener estado más tiempo $\rightarrow$ acepta tardíos pero requiere más estado



- **Micro-batches**

- Acumula eventos durante un intervalo y los procesa como bloque
- Simplifica fallos y permite obtener un alto throughput

- **Streaming nativo**

- Procesa de forma continua evento a evento
- Menor latencia y carga más uniforme sin esperas por lotes

- **Throughput y simplicidad vs latencia mínima**

Microprocesamiento por lotes

Esquema conceptual original de la edición abierta



- **Optimiza throughput y simplicidad operativa**
 - Trata el stream como secuencia de mini-datasets finitos
 - Simplifica recuperación ante fallos y checkpointing
- **A menudo integra batch y streaming**
 - API similar para batch y streaming
 - Aprovecha optimizaciones del motor batch
- **Ideal para analítica con latencias bajas**
 - Ejemplos incluyen Spark Structured Streaming

Motor Apache Spark

Esquema conceptual original de la edición abierta



- **Optimiza baja latencia evento a evento**
 - Procesa eventos de forma continua al llegar
 - Carga más uniforme sin esperas por lotes
- **Estado de primera clase gestionado por el motor**
 - Diseñado para event time, watermarks y estado distribuido
- **Ideal para latencias extremadamente bajas**
 - Ejemplos incluyen Apache Flink, Storm

Motor Apache Flink

Esquema conceptual original de la edición abierta

Motor Apache Storm

Esquema conceptual original de la edición abierta



- 1 Introducción
- 2 Mensajería distribuida
- 3 Procesamiento en streaming
- 4 Diseño a nivel de sistema**



- **Pasar de batch y monolitos a tiempo real distribuido supone un reto**
 - Nuevas piezas → mensajería distribuida y motor streaming
 - Nuevas propiedades → latencia, estado, tiempo de evento
 - Nuevas complejidades → desorden, tardíos, fallos parciales
- **El objetivo es equilibrar latencia, throughput y exactitud**



- **No es posible garantizar simultáneamente**

- **Consistency** → todos ven lo mismo a la vez
- **Availability** → el sistema siempre responde
- **Partition** → sobrevives a particiones de red

- **En sistemas distribuidos siempre hay particiones**

- Se debe elegir entre consistency (CP) o availability (AP)

Compromisos del teorema CAP

Esquema conceptual original de la edición abierta

<https://www.geeksforgeeks.org/system-design/>

[cap-theorem-in-system-design/](#)



- **AP prioriza disponibilidad y baja latencia**
 - Responder rápido aunque sea con una vista desactualizada
 - La consistencia eventual puede no estar al milisegundo
- **CP prioriza consistencia y exactitud**
 - Responder coherente aunque sea más lento
 - Puede bloquear o dar error antes que dar un dato inconsistente
- **Para analítica en vivo se suele priorizar AP**



- **Idealmente se quiere velocidad y exactitud a la vez**
 - Streaming da rapidez pero puede ser aproximado
 - Batch da precisión pero llega tarde
- **Lambda propone dos caminos en paralelo**
 - Capa rápida para resultados inmediatos
 - Capa batch para corregir el histórico
- **El usuario ve resultados rápidos que se refinan con el tiempo**



- **Speed layer para procesamiento en tiempo real**
 - Eventos recientes con baja latencia
 - Puede ser aproximado o tener gaps
- **Batch layer para reproceso del histórico completo**
 - Recalcula todo desde el principio periódicamente
 - Resultados exactos y completos pero que tardan más
- **Serving layer para combinar ambas vistas**
 - Speed para lo reciente + batch para el histórico corregido



Arquitectura Lambda

Esquema conceptual original de la edición abierta

<https://www.chaosgenius.io/blog/lambda-architecture/>



- **Proporciona velocidad y exactitud**
- **Supone una alta complejidad operativa**
 - Dos sistemas separados para batch y streaming
 - Misma lógica implementada dos veces
 - Difícil depurar cuando no coinciden los resultados
- **Lambda no vence el teorema CAP**



- **En Kappa todo es streaming**
 - Un solo motor para procesar eventos
 - Batch es simplemente un stream acotado con inicio y fin
- **Simplifica con un solo camino y una sola lógica**



- **Log persistente como fuente de verdad**
 - El sistema de mensajería conserva eventos
 - El motor de streaming lee desde el log
- **Procesamiento único con replay cuando se necesita**
 - Normalmente procesa eventos en tiempo real
 - Si hay que recalcular lee el log desde el principio (replay)
- **El motor de streaming procesa en vivo y reprocesa con replay**



Arquitectura Kappa

Esquema conceptual original de la edición abierta

<https://www.chaosgenius.io/blog/kappa-architecture/>



- **Si se descubre un error en el procesamiento**
 - Se corrige la lógica y se lanza un nuevo proceso
 - El proceso lee el log completo desde el principio
 - Procesa todo a máxima velocidad hasta alcanzar el presente
 - Se sustituye el proceso antiguo por el nuevo
- **Se requiere un log persistente y un motor streaming robusto**



- **Elegir entre Lambda o Kappa no es suficiente**
- **Existen múltiples aspectos clave a considerar**
 - ¿Cómo servir los datos según cada necesidad? (serving)
 - ¿Cómo operar el sistema en producción? (fiabilidad)
 - ¿Cómo garantizar calidad de datos en tiempo real?
 - ¿Cómo proteger privacidad y definir accesos?



- **Un mismo evento tiene diferentes vistas según el uso**
- **Dashboard en vivo (hot)**
 - KPIs por ventana (1–3 min) + alertas inmediatas
- **Histórico para ML y análisis (cold)**
 - Dataset curado para entrenar y validar modelos



- **El sistema debe manejar picos y fallos**
 - Durante un partido ocurren muchos eventos simultáneos
 - Si algo falla el sistema debe recuperar sin perder datos
- **Existen diferentes métricas clave a vigilar**
 - ¿Existen retrasados? (lag)
 - ¿Los datos llegan a tiempo? (latencia)
 - ¿Se están perdiendo eventos? (errores)
- **Si el sistema se satura se debe escalar o priorizar eventos**



- **En streaming no se puede parar para limpiar**
 - Se necesitan reglas claras de calidad
 - El sistema debe seguir funcionando con datos imperfectos
- **Se requiere validación y limpieza en tiempo real**
 - Eventos malformados van a cuarentena (dead-letter queue)
 - Deduplicar eventos repetidos por ID
 - Validar esquemas y compatibilidad
- **El reto no es solo ser rápido sino también coherente**



- **Es necesario definir quién accede a qué datos**
 - Diferentes roles con distintos permisos
 - Datos agregados vs datos individuales
- **Existen diferentes preguntas clave**
 - ¿Quién puede ver qué? (staff, analistas, médicos)
 - ¿Para qué se usa? (rendimiento, prevención, investigación)
 - ¿Cuánto tiempo se guarda? (retención y borrado)
- **Se deben definir responsabilidades y límites**



- **Elección de arquitectura según necesidades**
 - Batch si latencia no importa (reportes diarios)
 - Streaming si necesitas reaccionar rápido (alertas en vivo)
 - Lambda (robusta, dos caminos) vs Kappa (simple, un camino con replay)
- **Streaming consta de mensajería + motor + serving**
 - Mensajería que desacopla fuentes y consumidores
 - Motor streaming procesa con posible estado y ventanas de tiempo
 - Serving genera vistas hot (alertas en vivo) y cold (ML e histórico)
- **Trade-offs inevitables son velocidad vs exactitud vs complejidad**



- El texto, los ejemplos y los esquemas de esta edición abierta se distribuyen bajo CC BY-SA 4.0.
- Las imágenes de procedencia no acreditada de la versión de trabajo se han retirado y sustituido por esquemas conceptuales originales.
- Los logotipos y marcas citados se utilizan únicamente con finalidad identificativa y quedan excluidos de la licencia.
- Las referencias técnicas se conservan mediante citas y enlaces a sus fuentes correspondientes.
- Información de licencia: <https://creativecommons.org/licenses/by-sa/4.0/deed.es>

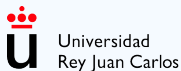
Big Data Processing II

Tema 2: Apache Kafka

Alberto Fernández Isabel
Natalia Madrueño Sierro
Rubén Rodríguez Fernández

DSLAB

Febrero 2026





Material docente en abierto de la Universidad Rey Juan Carlos

Asignatura: Big Data Processing II

Titulación: Máster Universitario en Análisis de Datos Deportivos

Autores: Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández

Fecha: 2026

© 2026 Equipo docente. Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional.

Los logotipos, marcas y referencias de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

Depósito institucional: <https://burjcdigital.urjc.es>



- Entender qué problema resuelve en el **análisis de datos deportivos**
- Conocer la arquitectura y los **conceptos clave** de Kafka
- Identificar **casos de uso** en datos deportivos en tiempo real
- Aprender **buenas prácticas** de diseño y operación de Kafka



- 1 **Introducción**
- 2 **Componentes**
- 3 **Infraestructura**
- 4 **Ejemplos**



- 1 **Introducción**
- 2 Componentes
- 3 Infraestructura
- 4 Ejemplos



- **Con pocas integraciones todo es simple**
 - Antes $\Rightarrow$ una aplicación alimenta directamente a otra
 - Ahora $\Rightarrow$ múltiples fuentes y múltiples destinos
- **La integración punto a punto introduce deuda técnica**
 - Cada fuente habla con cada destino
 - Formatos y contratos inconsistentes
 - Difícil rastrear el flujo de información end-to-end
 - Un cambio en un productor rompe consumidores en cascada
- **La complejidad crece rápidamente con cada nueva integración**

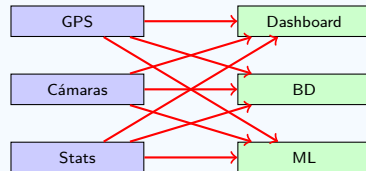


- **Sistema de datos en un estadio**

- Sensores GPS de jugadores
- Cámaras de tracking
- Sistema de estadísticas en vivo

- **El resultado es caos y complejidad**

- Acoplamiento fuerte entre sistemas
- n fuentes $\times$ m destinos = $n \times m$ integraciones



9 conexiones para 6 sistemas



- **Introducir un intermediario central**

- Productor → Distribuidor → Consumidor

- **Hub central de datos deportivos**

- Productores → clasifican y publican eventos
- Consumidores → se suscriben a lo que necesitan

- **Menos acoplamiento y mayor flexibilidad**

- Añadir consumidores no implica cambiar productores y viceversa
- Permite múltiples consumidores con diferentes necesidades

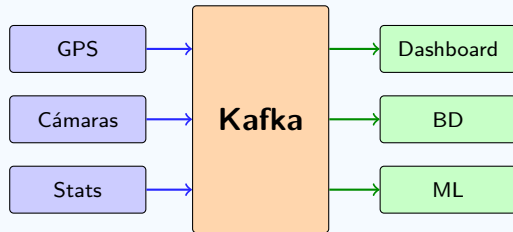


- **Kafka como hub central de mensajería**

- Todas las fuentes envían a Kafka
- Todas las herramientas leen de Kafka
- Los distribuye a múltiples destinos

- **Desacoplamiento entre sistemas**

- $n + m$ integraciones vs $n \times m$
- Sistemas independientes
- Formato estandarizado
- Escalabilidad horizontal



6 conexiones para 6 sistemas



- **Colas tradicionales**

- Diseñadas para distribución de tareas
- Mensajes eliminados tras consumo
- Un consumidor por mensaje

- **Logs persistentes**

- Diseñadas para análisis de datos y eventos
- Mensajes persistentes con retención configurable
- Múltiples consumidores independientes
- Replay de eventos históricos

- **Kafka es un commit log distribuido**



- **Desacopla fuentes (productores) de herramientas (consumidores)**
- **Múltiples consumidores para el mismo flujo de datos**
 - Dashboard en vivo para entrenadores
 - Análisis táctico con machine learning
 - Almacenamiento histórico en base de datos
- **Facilita análisis de datos deportivos en vivo**
 - Posiciones GPS de jugadores cada segundo
 - Eventos del juego, como goles, pases, tiros, tarjetas
 - Métricas fisiológicas, como ritmo cardíaco, velocidad



Arquitectura distribuida de referencia

Esquema conceptual original de la edición abierta

<https://www.geeksforgeeks.org/apache-kafka/kafka-architecture/>



- 1 Introducción
- 2 Componentes**
- 3 Infraestructura
- 4 Ejemplos



- **Estructura append-only**

- Escrituras secuenciales al final
- Inmutabilidad de eventos pasados
- Alta eficiencia de escritura y lectura
- Durabilidad de eventos configurable

- **Offsets como punteros de lectura**

- Identificador único por mensaje en partición
- Orden secuencial de eventos por partición
- Permite lectura determinista por partición
- Checkpointing del progreso del consumidor

Consumidor de eventos

Esquema conceptual original de la edición abierta



- **Estructura de un mensaje**

- **Key** (opcional) → ayuda a decidir partición y mantener orden por entidad
- **Value** → payload del mensaje en bytes
- **Timestamp** → marca temporal (por defecto del productor)

- **Kafka no impone un formato específico**

- JSON (recomendado)
- Apache Avro para consistencia



- Ejemplo de evento para gol en un partido

```
// Kafka Key liga_2025_j27_001, Timestamp 2025-03-15T16:42:18Z
{
  "schema_version": "1.0",
  "event_id": "3f7a2b9c-5d8e-4c1f-a6b3-9e4d2c1b5a7f",
  "event_time": "2025-03-15T16:42:18Z",
  "match_id": "liga_2025_j27_001",
  "event_type": "goal",
  "team_id": "real_madrid",
  "opponent_id": "barcelona",
  "scorer_id": "vinicius_jr",
  "assister_id": "modric",
  "minute": 67
}
```



- **Contrato que define la estructura de los datos**

- Plantilla que define qué campos tiene cada evento
- Facilita entendimiento entre productores y consumidores
- Evita errores → si falta un campo se detecta inmediatamente

- **Ejemplo de esquema para evento de gol**

- `event_id: string`
- `event_time: timestamp`
- `match_id: string`
- `event_type: string`
- `scorer_id: string`
- `assister_id: string`
- `minute: integer`



- **Lotes de mensajes**

- Colección de mensajes del mismo Topic y partición
- Reduce overhead de red y permite compresión
- En lugar de enviar 1 evento a la vez envía varios juntos
- Trade-off → lotes grandes pueden incrementar la latencia

- **Compresión de datos**

- Ahorra espacio y velocidad de red
- Kafka soporta compresión configurable
- Existen varios algoritmos de compresión



- **Canales lógicos donde se organizan los mensajes**
 - Categoría de mensajes relacionados
 - Organiza eventos por tipo o dominio
 - Patrón común de nombrado →
`dominio.subdominio.tipo.version`
- **Configuraciones clave**
 - Número particiones para paralelismo
 - Factor de replicación para durabilidad
 - Tiempo de retención de los mensajes
- **Ejemplo** `olympics.swimming.events.v1`

Particionado de un topic

Esquema conceptual original de la edición abierta



• **Procesamiento en paralelo**

- Un Topic se divide en múltiples particiones
- Unidad de paralelismo y escalado horizontal
- Cada partición es un log ordenado independiente
- Es *append-only* → se escribe al final

• **Orden garantizado dentro de una partición**

- No garantiza orden en todo el Topic
- Key → misma partición
- Sin key → round-robin o sticky

Particionado de un topic

Esquema conceptual original de la edición abierta



- **Productores**

- Escriben mensajes en Topics
- Balancean carga entre particiones
- Con *key* garantizan orden por entidad en una partición

- **Consumidores**

- Leen mensajes manteniendo orden dentro de cada partición
- Mantienen offsets con la posición del último mensaje leído
- Para recuperación tras fallo reinician desde el último offset guardado



- **Marcadores de lectura en partición**

- Permite recordar qué eventos se han procesado
- Como un marcador en un libro

- **Ejemplo práctico de uso de offsets**

- Se disponen 1000 eventos GPS
- La lectura se cae en el evento 750
- Al reiniciar se continúa desde el 751

- **Commit automático vs manual**

- En *Auto* → Guarda el offset automáticamente
- En *Manual* → Tú decides cuándo guardarlo

Gestión de offsets

Esquema conceptual original de la edición abierta



- **Indica qué tan atrasado está el consumidor**

- Diferencia entre ultimo offset disponible y offset del consumidor
- End Offset=1000, Current Offset=750 $\rightarrow$ Lag=250 mensajes

- **Importancia de monitorear el lag**

- Lag alto $\rightarrow$ consumidor lento o problemas de procesamiento
- Lag creciente $\rightarrow$ consumidor no puede mantener el ritmo
- Alertar cuando supera umbrales críticos



- **Un Topic puede ser leído por múltiples consumidores**
- **Mismo grupo de consumidores**
 - Múltiples consumidores procesando en paralelo un Topic
 - Si un consumidor procesa rápido, reduce el lag del grupo
 - Si un consumidor falla, otros continúan
 - Ejemplo → 3 consumidores procesando eventos GPS
- **Único consumidor (distinto grupo)**
 - Aplicaciones independientes
 - Ejemplo → Dashboard + Análisis ML + DB histórica



- **Permite paralelizar la lectura de un Topic**
 - Escalabilidad horizontal de consumidores
 - Consumidores agrupados por `group_id`
 - Paraleliza lectura dentro de un Topic
- **Los mensajes se distribuyen en particiones**
 - Dentro del grupo cada partición la lee un solo consumidor a la vez
 - Para rendimiento máximo de lectura $\rightarrow$ particiones $\approx$ consumidores (en el grupo)
- **Si un consumidor falla los restantes se reparten sus particiones**

Grupo de consumidores

Esquema conceptual original de la edición abierta



- **Reasignación automática de particiones**
 - Se activa cuando un consumidor se une, sale o falla
 - Las particiones se redistribuyen entre consumidores activos
- **Impacto durante el rebalance**
 - El grupo deja de consumir temporalmente
 - Puede causar duplicados si no se commitean offsets antes
 - Tiempo de pausa depende del tamaño del grupo
- **Varias estrategias de asignación**
 - Range, Round-robin, Sticky



- 1 Introducción
- 2 Componentes
- 3 Infraestructura**
- 4 Ejemplos



- **Broker**

- Almacena particiones asignadas
- Es un servidor individual (nodo) del clúster
- Tienen un identificador único

- **Clúster**

- Conjunto de brokers coordinados
- Un broker actúa como controlador
- Distribución de particiones entre brokers

- **Permite escalabilidad y alta disponibilidad**



- **Modelo líder-seguidor**

- Líder maneja lecturas y escrituras
- Seguidores copian log del líder

- **In-Sync Replicas (ISR)**

- Réplicas sincronizadas con el líder
- Pueden ser elegidas como líder

- **Si el líder falla se elige otro**

- Nuevo líder es un seguidor en ISR
- El sistema sigue funcionando sin pérdida

Replicación y tolerancia a fallos

Esquema conceptual original de la edición abierta



- **Acknowledgment (ack)**

- Recibido por el productor
- Brokers confirman mensajes recibidos

- **Tipos de acknowledgment**

- `acks=0` → sin confirmación
- `acks=1` → líder confirma
- `acks=all` → todas las ISR confirman



- **At-most-once**

- El dato puede perderse pero no se duplica
- Cuando velocidad es más importante que completitud
- Ejemplo → Sensor GPS falla, pierdes una posición

- **At-least-once** - Opción más común

- El dato llega pero puede duplicarse
- Garantiza que no se pierden eventos
- Ejemplo → El mismo gol aparece dos veces en tu sistema

- **Exactly-once**

- El dato se procesa una vez
- Kafka lo soporta pero requiere configuración avanzada



- **Durabilidad es no perder datos**
- **Tres pilares para garantizar durabilidad**
 - Producers esperan confirmación del broker
 - Múltiples copias en ISR
 - Datos escritos a disco
- **Mayor durabilidad implica mayor latencia**



- **Zookeeper (legacy)**

- Almacena metadatos del clúster y coordina elección de líderes
- Ensemble con número impar de nodos (3, 5, 7)
- Requiere gestión adicional de infraestructura

- **KRaft (Kafka 3.0+)**

- Gestiona metadatos y líderes sin Zookeeper
- Protocolo Raft para consenso y elección de líderes
- Menos complejidad y arranque más rápido

- **Se recomienda KRaft para nuevos clústeres**

Coordinación mediante KRaft

Esquema conceptual original de la edición abierta



- **Regla práctica**

- Particiones $\approx$ **throughput deseado** / **velocidad del consumidor**
- Ejemplo $\rightarrow 1 \text{ GB/s} / 50 \text{ MB/s} \approx 20$ particiones

- **Consideraciones**

- Se pueden aumentar, pero no disminuir
- Más particiones $\rightarrow$ más concurrencia
- Demasiadas $\rightarrow$ mayor uso de memoria y coste de coordinación



- **Retención por tiempo**

- Se borran segmentos cuando el último mensaje supera la edad límite
- Ejemplo → conservar datos de un partido durante 7 días

- **Retención por tamaño**

- Útil para evitar llenar el disco con flujos impredecibles
- Ejemplo → máximo 1GB por partición

- **Log compaction**

- Mantiene el último valor por clave
- Elimina versiones antiguas de la misma clave
- Ejemplo → Mantener última posición GPS de cada jugador



- 1 Introducción
- 2 Componentes
- 3 Infraestructura
- 4 Ejemplos**



- **Kafka 4.1+ incluye KRaft (sin Zookeeper)**
 - Arranque rápido con un solo contenedor
 - Ideal para desarrollo y pruebas locales
- **Comando para lanzar Kafka localmente**
 - Puerto 9092 expuesto para productores y consumidores
 - Broker disponible en localhost:9092

Terminal Shell Terminal

```
docker run -p 9092:9092 apache/kafka:4.1.1
```



Terminal Shell Terminal

```
docker run -p 9092:9092 \  
  -e KAFKA_HEAP_OPTS="-Xmx1G -Xms1G" \  
  -e KAFKA_LOG_RETENTION_HOURS=168 \  
  -e KAFKA_NUM_PARTITIONS=3 \  
  -e KAFKA_DEFAULT_REPLICATION_FACTOR=1 \  
  -e KAFKA_COMPRESSION_TYPE=gzip \  
  apache/kafka:4.1.1
```

Memoria JVM
Retencion 7 dias
Particiones defecto
Factor replicacion
Compresion



```
services:
  kafka:
    image: apache/kafka:4.1.1
    ports:
      - "9092:9092"
    volumes:
      - kafka-data:/var/lib/kafka/data      # Persistencia
    environment:
      KAFKA_HEAP_OPTS: "-Xmx1G -Xms1G"     # Memoria JVM
      KAFKA_LOG_RETENTION_HOURS: 168        # Retencion 7 dias
      KAFKA_NUM_PARTITIONS: 3               # Particiones defecto
      KAFKA_DEFAULT_REPLICATION_FACTOR: 1  # Factor replicacion
      KAFKA_COMPRESSION_TYPE: gzip          # Compresion
```



- **Librería kafka-python para casos de uso sencillos**
 - Cliente básico de Kafka para Python
 - Soporte completo para Producer y Consumer
- **Componentes principales**
 - `KafkaProducer`: Envía mensajes a topics
 - `KafkaConsumer`: Lee mensajes de topics
 - `KafkaAdminClient`: Gestión de topics y configuración
- **Librería confluent-kafka para producción y alto rendimiento**



Python Python Code

```
from kafka import KafkaProducer
import json
import random
import time

producer = KafkaProducer(
    bootstrap_servers=['localhost:9092'],
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

for i in range(100):
    message = {'id': i, 'value': random.randint(1, 100)}
    producer.send('random.numbers.v1', value=message)
    time.sleep(1)

producer.close()
```



Python Python Code

```
producer = KafkaProducer(  
    bootstrap_servers=['localhost:9092'],  
    value_serializer=lambda v: json.dumps(v).encode('utf-8'),  
  
    # Garantías de entrega  
    acks='all',                # 0, 1, o 'all'  
    retries=3,                 # reintentos  
    enable_idempotence=True,   # exactly-once  
  
    # Rendimiento  
    compression_type='gzip',   # gzip, snappy, lz4, zstd  
    batch_size=16384,          # tamaño lote (bytes)  
    linger_ms=10               # espera antes de enviar  
)
```



Python Python Code

```
from kafka import KafkaConsumer
import json

consumer = KafkaConsumer(
    'random.numbers.v1',
    bootstrap_servers=['localhost:9092'],
    auto_offset_reset='latest',
    value_deserializer=lambda m: json.loads(m.decode('utf-8'))
)

print("Listening for messages...")

for message in consumer:
    data = message.value
    print(f"Received: ID={data['id']}, Value={data['value']}")
```



Python Python Code

```
consumer = KafkaConsumer(  
    'random.numbers.v1',  
    bootstrap_servers=['localhost:9092'],  
  
    # Deserialización de mensajes  
    key_deserializer=lambda k: k.decode('utf-8') if k else None,  
    value_deserializer=lambda m: json.loads(m.decode('utf-8')),  
  
    # Grupo de consumidores  
    group_id='my-consumer-group',  
  
    # Gestión de offsets  
    auto_offset_reset='latest',      # earliest, latest, none  
)
```




- **Kafka como hub central de datos deportivos**
 - Reduce complejidad y fragilidad frente a integraciones P2P
 - Hub central para datos deportivos en tiempo real
- **Kafka es un commit log distribuido**
 - Persistencia y retención → replay y múltiples consumidores
 - Topics/particiones → orden por partición y paralelismo
 - Offsets → control del progreso y recuperación ante fallos
- **Operación y escalado requieren diseño**
 - Replicación líder/seguidores, dimensionamiento de particiones
 - Políticas de retención según tipo de dato deportivo
 - Configuración apropiada para garantías de entrega



- El texto, los ejemplos y los esquemas de esta edición abierta se distribuyen bajo CC BY-SA 4.0.
- Las imágenes de procedencia no acreditada de la versión de trabajo se han retirado y sustituido por esquemas conceptuales originales.
- Los logotipos y marcas citados se utilizan únicamente con finalidad identificativa y quedan excluidos de la licencia.
- Las referencias técnicas se conservan mediante citas y enlaces a sus fuentes correspondientes.
- Información de licencia: <https://creativecommons.org/licenses/by-sa/4.0/deed.es>

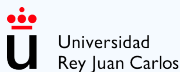
Big Data Processing II

Tema 2: Spark en Streaming

Alberto Fernández Isabel
Natalia Madrueño Sierro
Rubén Rodríguez Fernández

DSLAB

Febrero 2026





Material docente en abierto de la Universidad Rey Juan Carlos

Asignatura: Big Data Processing II

Titulación: Máster Universitario en Análisis de Datos Deportivos

Autores: Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández

Fecha: 2026

© 2026 Equipo docente. Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional.

Los logotipos, marcas y referencias de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

Depósito institucional: <https://burjcdigital.urjc.es>



- Entender qué problema resuelve en el **análisis de datos deportivos**
- Conocer los conceptos clave de **Spark Structured Streaming**
- Aplicar **casos de uso** en datos deportivos en tiempo real
- Saber **optimizar consultas** y rendimiento en streaming



- 1 **Introducción**
- 2 **Componentes**
- 3 **Infraestructura**
- 4 **Ejemplos**



- 1 **Introducción**
- 2 Componentes
- 3 Infraestructura
- 4 Ejemplos



- **Procesamiento de datos en tiempo real**
 - Datos que llegan continuamente (logs, sensores, eventos)
 - Procesamiento incremental a medida que llegan
 - Latencia baja (segundos) vs batch (horas/días)
- **Structured Streaming proporciona una API unificada**
 - Misma API que en batch (DataFrames/SQL)
 - Spark SQL se encarga de la gestión del streaming
 - Garantías de consistencia y tolerancia a fallos
- **Para monitorización en tiempo real, alertas, ETL continuo**



- **Spark DStream (legacy)**
 - Basado en RDDs y micro-batching rígido
 - Opera a nivel de tiempo de proceso
 - API de más bajo nivel
- **Structured Streaming (recomendado)**
 - Basado en DataFrames/Datasets
 - Soporte nativo en tiempo de evento
 - Optimizado por Catalyst Optimizer
- **Se trabajará con Spark Structured Streaming**



- **Structured Streaming tiene dos modos**
- **Micro-batch (por defecto)**
 - Ejecuta el stream como una secuencia de micro-batches
 - Compatibilidad con la mayoría de operadores y destinos
 - Latencia de ~ 100 ms (mínimo) a segundos
- **Continuous processing (experimental)**
 - Pensado para latencias muy bajas
 - Menos operadores soportados y más restricciones
 - Latencia de ~ 1 ms



- **Un stream es una tabla que crece continuamente**
 - Cada evento nuevo es una fila que se añade
 - Una consulta genera resultados de forma incremental
- **Un flujo continuo de eventos es una tabla infinita**

Modelo tabular de streaming

Esquema conceptual original de la edición abierta



- **Se programa como en procesamiento en batch**
 - API SQL/DataFrame con transformaciones encadenadas
 - Misma lógica para datos finitos y para flujos continuos
- **Modelo mental: *tabla* → *query* → *salida***
 - El *stream* se interpreta como una tabla que crece
 - La query produce resultados de forma incremental
- **Enfoque declarativo**
 - Defines *qué* calcular, no los detalles de ejecución
 - Spark planifica y optimiza el *cómo*



- **Misma API que en batch**

- Operaciones habituales (`select`, `join`, `count`, ...)
- Permite reutilizar lógica batch en pipelines de streaming

- **Particularidades en streaming**

- Resultados incrementales → la salida se actualiza con nuevos datos
- Gestión de estado → agregaciones/ventanas
- Semánticas de salida → *append*, *update*, *complete*



- **Consulta declarativa sobre un stream de eventos**
 - Se expresa como una query de tipo batch (DataFrame/SQL)
 - Se evalúa incrementalmente en cada trigger con los datos nuevos
 - Spark SQL la planifica y optimiza (Catalyst + Tungsten)
- **Los resultados se escriben en un destino de salida**
- **Ejemplos son resultados que se actualizan en cada trigger**



Ejemplo de conteo continuo

Esquema conceptual original de la edición abierta



- 1 Introducción
- 2 Componentes**
- 3 Infraestructura
- 4 Ejemplos



Arquitectura de Structured Streaming

Esquema conceptual original de la edición abierta

Entrada → Transformaciones (sin/con estado) → Salida



Flujo de Structured Streaming

Esquema conceptual original de la edición abierta



- **SparkSession es el punto de entrada**
 - Las entradas (sources) se leen con `spark.readStream`
 - Las salidas (sinks) se escriben con `df.writeStream`
- **Del mismo stream se pueden ejecutar múltiples queries**
 - Query 1 → resultados por ventana (como posesión) para un dashboard
 - Query 2 → detecta eventos críticos (posible lesión) y los publica en Kafka



- **Lectura de streams con `spark.readStream`**
- **Soporta múltiples fuentes habituales**
 - Kafka como sistema de mensajería
 - File para nuevos ficheros en un directorio
 - Socket para pruebas rápidas (Netcat)
 - Rate para testing y benchmarking
- **El paralelismo suele venir de la fuente**
 - Como particiones de tópicos de Kafka
 - Más particiones $\Rightarrow$ más tareas en paralelo

Plataforma de eventos Kafka

Esquema conceptual original de la edición abierta



- **Control de ejecución de una query**

- `query = df.writeStream... .start()` inicia una query
- `query.awaitTermination()` bloquea hasta que termine
- `query.stop()` detiene la query (graceful stop)

- **Operación de queries en producción**

- Poner un nombre descriptivo a la query
- Un mismo job (programa) puede ejecutar múltiples queries en paralelo



- **En streaming el esquema es esencial**
 - En un stream infinito no se recomienda inferir
 - Es importante definir el esquema (JSON)
 - Esto incluye tipar los campos de eventos
- **Decisiones de parsing de eventos**
 - Registros corruptos → enviar a cola aparte o fallar
 - Evolución de esquema → versionado/validación



- **Operaciones sin recordar el pasado**

- Suelen escalar mejor → no requieren estado compartido entre micro-batches
- Algunas son `select`, `filter`, `withColumn`, `explode`, `map`, ...
- Filtrar `event_type == "SHOT"` o extraer minutos de un timestamp

- **Si no hay agrupación/ventanas/join temporal/dedup normalmente es stateless**



- **Matienen el estado entre micro-batches**
 - Recuerdan eventos anteriores
- **Tipos de operaciones con estado**
 - Agregaciones → `groupBy`, `count/sum/avg`
 - Deduplicación → `dropDuplicates`
 - Joins → `stream-static` y `stream-stream`
 - Estado custom → `mapGroupsWithState`



- **Propiedades a alto nivel**

- Estado actualizado incrementalmente entre micro-batches
- Recuperación y tolerancia a fallos mediante checkpointing
- Limpieza y estado acotado con watermarks o timeouts

- **Ejemplo → resultados por ventanas de últimos 5 minutos**



- **Stream-to-Static**

- Enriquecer un stream con datos (relativamente) estáticos
- Ejemplo → eventos del partido en streaming + datos de la plantilla

- **Stream-to-Stream**

- Correlacionar eventos entre dos streams
- Requiere tiempo de evento + watermarks + rango temporal para acotar estado
- Ejemplo → eventos del partido + biometría en streaming

- **Joins stream-stream sin cota temporal crecen el estado indefinidamente**



- **Eventos tardíos / fuera de orden**
 - Cuando ocurrió $\neq$ Cuando llegó
 - Ventanas/agregaciones usan tiempo de evento
 - El trigger marca cada cuánto se procesa
- **Watermark define la tardanza aceptada**
 - Cierra ventanas y acota estado
 - Define hasta cuándo se procesan eventos tardíos



- **Agrupaciones en intervalos de tiempo**
- **Ventanas fijas (tumbling)**
 - Bloques fijos y no superpuestos
- **Ventanas deslizantes (sliding)**
 - Bloques que se superponen
- **Ventanas de sesión (session)**
 - Separados por gap de inactividad
- **Es una columna más de agrupación**

Ventanas temporales

Esquema conceptual original de la edición abierta



Agregación por ventana

Esquema conceptual original de la edición abierta



- **Define cada cuánto se ejecuta la query para procesar nuevos datos**
 - Default → tan rápido como sea posible (micro-batch)
 - ProcessingTime → intervalo fijo (por ejemplo cada 10s)
 - AvailableNow → procesa lo disponible y se detiene
 - Continuous → sólo en modo continuous (con restricciones)



- **Define cuánto esperar un evento tardío**
 - Los eventos no siempre llegan en orden ni a tiempo
 - El GPS de un jugador puede llegar 2 minutos tarde al servidor
 - El watermark define cuánto retraso toleramos antes de cerrar una ventana
- **Su uso es fundamental para controlar el estado**
 - Un watermark de 2 minutos (tiempo de evento) acepta tardíos hasta 2 min
 - Más allá los tardíos no actualizan ventanas/estado
- **Supone un trade-off**
 - Watermark amplio → más precisión pero más memoria usada
 - Watermark corto → resultados más rápidos pero posibles eventos perdidos



- **Es habitual que lleguen eventos tardíos**

- Dentro del watermark → se incluye en el cálculo y puede corregir un resultado emitido
- Fuera del watermark → se descarta para mantener los resultados estables

- **Ejemplo → tiros a puerta en los últimos 5 minutos**

- Un sensor registra un tiro a las 18:32 pero el dato llega a las 18:34
- Con watermark de 3 minutos → se acepta y corrige del intervalo 18:30–18:35
- Con watermark de 1 minuto → llega demasiado tarde y se descarta



Gestión de datos tardíos

Esquema conceptual original de la edición abierta



- **Append escribe solo filas nuevas y finales**
 - No reescribe filas
 - En ventanas emite cuando la ventana se “cierra”
 - Útil para eventos crudos o resultados finales
- **Update escribe filas que cambian**
 - Reescribe solo las filas actualizadas desde el último trigger
 - Útil para resultados en vivo (contadores en evolución)
- **Complete reescribe la tabla completa en cada trigger**
 - Solo viable si la tabla de resultados es pequeña



Watermark en modo append

Esquema conceptual original de la edición abierta



Watermark en modo update

Esquema conceptual original de la edición abierta



- **Escritura de resultados con `writeStream`**
- **Soporta múltiples destinos habituales**
 - Sistemas de mensajería (Kafka)
 - Bases de datos SQL y NoSQL (InfluxDB)
 - Archivos como HDFS/S3 (Parquet/JSON)
 - Consola/memoria para depuración y desarrollo
 - `foreach/foreachBatch` para lógica personalizada

Visualización operativa

Esquema conceptual original de la edición abierta



- 1 Introducción
- 2 Componentes
- 3 Infraestructura**
- 4 Ejemplos



- **Eliminar duplicados en streaming**

- La fuente puede reemitir eventos (retries, reconexiones, *at-least-once*)
- Se deduplica por una clave estable (como `event_id`)
- Se usa watermark → limita cuánto tiempo recordamos



- **Las operaciones con estado consumen recursos**
 - Spark necesita recordar eventos pasados para calcular agregaciones y ventanas
 - Más claves de agrupación o ventanas más amplias $\Rightarrow$ más memoria necesaria
- **El watermark limita cuánto estado se acumula**
 - Permite a Spark olvidar ventanas ya cerradas
 - Sin watermark el estado crece indefinidamente
- **Regla práctica: agrupa por pocas claves y usa ventanas razonables**



- **El trigger controla cada cuánto se actualizan los resultados**
 - Trigger corto → datos más frescos pero más carga en el sistema
 - Trigger largo → datos menos frescos pero menos carga
- **Regla práctica: elige el trigger según la necesidad del análisis**
 - Dashboard en tiempo real → trigger de segundos
 - Reporte por minuto → trigger de 60s es suficiente



- **Spark recuerda hasta dónde llegó**
 - Guarda un punto de control tras cada micro-batch
 - Si el proceso cae, retoma desde ahí sin reprocesar datos ya calculados
 - Evita duplicados y pérdida de resultados intermedios
- **Checkpointing para recuperación tras fallos**
 - Persiste progreso y estado
 - Spark persiste logs de progreso/commits asociados a cada trigger
 - Tras un reinicio reanuda desde el último progreso consistente



- **Spark garantiza que cada evento se procesa exactamente una vez**
 - Los resultados no se duplican ni se pierden aunque el sistema falle y se reinicie
- **Para que funcione se necesitan tres condiciones**
 - La fuente permite releer datos (como Kafka)
 - Spark guarda el progreso (checkpointing)
 - El destino tolera escrituras duplicadas sin problemas



- 1 Introducción
- 2 Componentes
- 3 Infraestructura
- 4 Ejemplos**



- Se recomienda despliegue local / Docker para pruebas
- **Requisitos de instalación en despliegue local de Spark**
 - Java instalado (`java --version`) y configurar `JAVA_HOME`
 - Instalar PySpark y crear una sesión local con todos los núcleos [*]

Terminal Shell Terminal

```
java --version           # verificar Java
apt install -y openjdk-21-jdk # instalar Java (Colab / Linux)
pip install pyspark      # instalar PySpark
```



- Si `JAVA_HOME` no está definida hay que indicarla manualmente

Python Python Code

```
import os
import pyspark

# En Debian/Ubuntu con version 21 de Java:
# os.environ["JAVA_HOME"] = "/usr/lib/jvm/java-21-openjdk-amd64"
# En Windows con version 21 de Java:
# os.environ["JAVA_HOME"] = "C:\\Program Files\\Java\\jdk-21"

print(os.environ["JAVA_HOME"]) # comprobar si esta definida
print(pyspark.__version__)
```



- Jupyter + PySpark ya instalados y configurados

Terminal Shell Terminal

```
docker run \  
  --network=host \  
  -e JUPYTER_TOKEN="MY_SECRET" \  
  -v ${PWD}:/home/jovyan/work \  
  jupyter/pyspark-notebook
```



```
services:
  jupyter:
    image: jupyter/pyspark-notebook
    network_mode: host
    environment:
      - JUPYTER_TOKEN=MY_SECRET
    restart: on-failure
    volumes:
      - ./:/home/jovyan/work
```




❶ Leer líneas Spark desde un socket en el puerto 9999

- Cada línea representa un evento de un partido de fútbol
- Pueden ser goal, shot, foul y pass

❷ Filtrar solo eventos válidos y normalizar cada línea

❸ Agregar el conteo acumulado por tipo de evento

❹ Escribir los resultados en consola



- **Paso 0 - Simular un flujo de tipos de eventos de un partido de fútbol**
 - Socket con Netcat para enviar eventos de prueba
 - En Windows instalar Nmap (<https://nmap.org/download>)

Terminal Shell Terminal

En local

```
nc -lv 9999
```

En docker

```
docker run --rm -it --init -p 9999:9999 madrueno/nc -lv 9999
```



• Paso 1 - Iniciar una sesión de Spark Structured Streaming

Python Python Code

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lower, trim

spark = (
    SparkSession.builder                # Construye la sesión de Spark
    .appName("FootballStreamingEventCount") # Nombre descriptivo
    .getOrCreate()                      # Crea o reutiliza la sesión existente
)
```



- Paso 2 - Leer líneas de eventos desde el socket abierto

Python Python Code

```
lines = (  
    spark.readStream                # Lee un stream de una fuente  
    .format("socket")              # Fuente de tipo socket  
    .option("host", "localhost")   # Host del socket  
    .option("port", 9999)          # Puerto del socket  
    .load()                        # Carga como DataFrame  
)
```



• Paso 3 - Normalizar texto y filtrar eventos válidos

- Convertir a minúsculas y eliminar espacios
- Filtrar solo los tipos de eventos de interés

Python Python Code

```
valid = ["goal", "shot", "foul", "pass"]           # Tipos de eventos válidos

events = (
    lines                                           # DataFrame (string por defecto)
    .select(trim(lower(col("value"))).alias("event")) # Normaliza texto
    .where(col("event").isin(valid))               # Filtra solo eventos válidos
)
```



● Paso 4 - Agregar el conteo acumulado por tipo de evento

- Agrupa por tipo y cuenta el número de eventos por tipo
- La query no se ejecuta hasta que se escriba la salida

Python Python Code

```
counts = (  
    events                # DataFrame con eventos  
    .groupBy("event")     # Agrupa por tipo de evento  
    .count()              # Cuenta el número de eventos por tipo  
)
```



- Paso 5 - Escribir resultados continuamente en consola

Python Python Code

```
query = (  
    counts.writeStream          # Define el destino de salida  
    .outputMode("complete")    # Muestra la tabla completa de conteos  
    .format("console")         # Escribe en consola  
    .start()                   # Inicia la query de streaming  
)  
  
query.awaitTermination()      # Espera hasta que se detenga la query
```



• Entradas de prueba enviadas por netcat

- Eventos con timestamp, equipo, tipo de evento y xG
- Se define un esquema explícito para parsear el JSON
- El campo ts permite aplicar ventanas temporales y watermark

Terminal Shell Terminal

```
{"ts": "2025-06-01 10:00:10", "team": "RMA", "event": "goal", "xg": 0.9}  
{"ts": "2025-06-01 10:00:40", "team": "FCB", "event": "shot", "xg": 0.2}  
{"ts": "2025-06-01 10:02:55", "team": "RMA", "event": "shot", "xg": 0.3}  
{"ts": "2025-06-01 09:45:40", "team": "FCB", "event": "shot", "xg": 0.2}
```




Python Python Code

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.types import (
    StructType, StructField,
    StringType, DoubleType, TimestampType
)

spark = (
    SparkSession.builder
    .appName("FootballStreamingGoalEvents")
    .getOrCreate()
)
```



- **Definir el esquema explícito del evento JSON**

- Cada línea del socket es un objeto JSON con 4 campos

Python Python Code

```
schema = StructType([
    StructField("ts",    TimestampType()), # Timestamp del evento
    StructField("team",  StringType()),     # Equipo que realizó la acción
    StructField("event", StringType()),     # Tipo de evento (goal, shot, etc.)
    StructField("xg",    DoubleType()),     # Goles esperados del evento
])
```



- **Leer y parsear eventos JSON desde el socket**
 - Cada línea recibida se parsea con el esquema definido

Python Python Code

```
events = (  
    spark.readStream  
        .format("socket")  
        .option("host", "localhost")  
        .option("port", 9999)  
        .load() # Columna "value" con el JSON crudo  
        .select(F.from_json("value", schema).alias("e")) # Parsea JSON con el schema  
        .select("e.*") # Desempaqueta los campos  
)
```



- **Aplicar watermark y agregar en ventanas**

- Tomamos ventanas de 1 minuto a modo de ejemplo
- El watermark descarta eventos que llegan con más de 2 minutos de retraso
- La ventana fija agrupa en bloques de 1 minuto según el timestamp del evento

Python Python Code

```
counts = (  
    events  
    .filter(F.col("event").isin("goal", "shot")) # Filtra solo acciones de gol  
    .withWatermark("ts", "5 minutes")           # Descarta eventos muy tardíos  
    .groupBy(F.window("ts", "1 minute"), "team") # Ventana de 1 min por equipo  
    .agg(F.count("*").alias("total"))           # Cuenta acciones por ventana  
)
```



- **Escribir resultados en consola con trigger periódico**

- El trigger actualiza los resultados cada 10 segundos

Python Python Code

```
query = (  
    counts.writeStream(  
        .outputMode("update")           # Emite resultados actualizados  
        .format("console")  
        .trigger(processingTime="10 seconds") # Ejecuta un micro-batch cada 10 s  
        .start()  
    )  
  
    query.awaitTermination()
```



- Enviar eventos con timestamps desordenados

Terminal Shell Terminal

Batch 1 de eventos

```
{"ts": "2025-06-01 10:00:10", "team": "RMA", "event": "goal", "xg": 0.9}  
{"ts": "2025-06-01 10:00:40", "team": "FCB", "event": "shot", "xg": 0.2}  
{"ts": "2025-06-01 10:02:55", "team": "RMA", "event": "shot", "xg": 0.3}
```

Terminal Shell Terminal

Evento tardío (llega después pero ocurrió antes)

```
{"ts": "2025-06-01 09:45:40", "team": "FCB", "event": "shot", "xg": 0.2}
```



- **Spark Structured Streaming para análisis en tiempo real**
 - Unifica batch y streaming con un modelo declarativo
 - Actualiza resultados incrementalmente y gestiona estado, tardíos
- **Checklist práctico de desarrollo**
 - Establecer fuente y definir el esquema de eventos
 - Identificar tiempo de evento y configurar watermarks/ventanas
 - Escoger destino y modo de escritura según necesidad
 - Definir trigger para controlar frescura de resultados
 - Ejecutar `start()` y `awaitTermination()`



- El texto, los ejemplos y los esquemas de esta edición abierta se distribuyen bajo CC BY-SA 4.0.
- Las imágenes de procedencia no acreditada de la versión de trabajo se han retirado y sustituido por esquemas conceptuales originales.
- Los logotipos y marcas citados se utilizan únicamente con finalidad identificativa y quedan excluidos de la licencia.
- Las referencias técnicas se conservan mediante citas y enlaces a sus fuentes correspondientes.
- Información de licencia: <https://creativecommons.org/licenses/by-sa/4.0/deed.es>

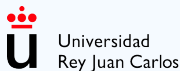
Big Data Processing II

Tema 3: Optimización y Escalabilidad de Pipelines Deportivos

Alberto Fernández Isabel
Natalia Madrueño Sierro
Rubén Rodríguez Fernández

DSLAB

Febrero 2026





Material docente en abierto de la Universidad Rey Juan Carlos

Asignatura: Big Data Processing II

Titulación: Máster Universitario en Análisis de Datos Deportivos

Autores: Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández

Fecha: 2026

© 2026 Equipo docente. Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional.

Los logotipos, marcas y referencias de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

Depósito institucional: <https://burjcdigital.urjc.es>



- Conocer **buenas prácticas** de diseño de pipelines de análisis de datos
- Identificar técnicas de **optimización y escalabilidad** en Kafka y Spark
- Reconocer opciones de infraestructura en producción **on-premises y cloud**



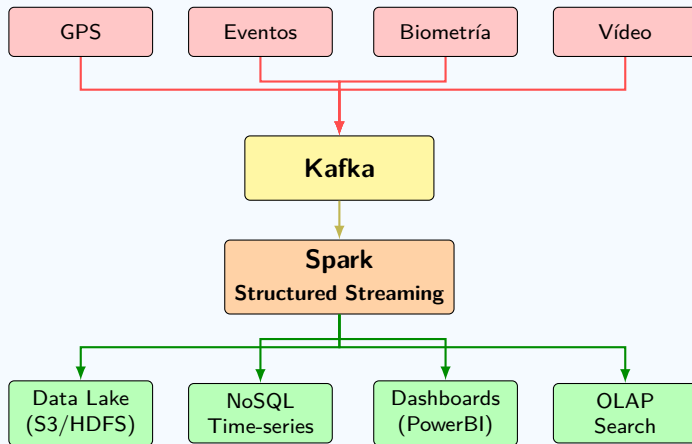
- 1 **Introducción**
- 2 **Buenas prácticas**
- 3 **Puesta en producción**



- 1 **Introducción**
- 2 Buenas prácticas
- 3 Puesta en producción



- **Múltiples fuentes de datos en vivo** → Como tracking, sensores, eventos y vídeo
- **Kafka para mensajería distribuida** → Bus de ingestión y retención de eventos
- **Spark para análisis de datos** → Tanto en batch como en streaming
- **Múltiples salidas operativas** → Como data lakes, BBDD y dashboards





Arquitectura distribuida de referencia

Esquema conceptual original de la edición abierta

Fuente: <https://www.scaler.com/topics/spark-streaming-with-kafka/>



- **Rapidez en el análisis**

- Respuesta en tiempo real durante el partido
- Alertas y apoyo a decisiones en vivo

- **Capacidad y fiabilidad del sistema**

- Procesamiento de grandes volúmenes de datos deportivos
- Integración de dispositivos tracking, eventos y sensores
- Funcionamiento estable en momentos críticos de competición

- **Los objetivos marcan la arquitectura y los recursos**

- SLA/SLO claros para guiar decisiones
- Latencia objetivo y disponibilidad esperada



- **El cuello de botella se desplaza entre componentes**

- Optimizar un componente no garantiza mejorar todo el pipeline
- Al mejorar una parte la limitación suele aparecer en otra
- La optimización debe evaluarse a nivel global

- **La monitorización es imprescindible**

- Comparar métricas antes y después de cada cambio
- Monitorizar retrasos en Kafka y tiempos en Spark
- Vigilar uso de recursos como disco, red y memoria



- **Kafka escala mejor cuando**

- Hay suficientes particiones para paralelizar producción y consumo
- Se añaden brokers para repartir carga y almacenamiento
- La infraestructura ofrece buena red y discos rápidos

- **Spark escala mejor cuando**

- Hay suficiente particionado para el paralelismo de tareas
- Se asignan executors y memoria adecuados
- Se reducen los shuffles innecesarios



- 1 Introducción
- 2 Buenas prácticas**
- 3 Puesta en producción



- **Gobierno y estabilidad del dato**

- Mantener un esquema estable en pipelines críticos
- Definir reglas de evolución para evitar cambios incompatibles

- **Organización del almacenamiento**

- Elegir un particionado coherente con las consultas más frecuentes
- Compactar periódicamente para mejorar el rendimiento
- Un buen diseño mejora el rendimiento de lectura y escritura

- **Diseño según coste, latencia y tipo de procesamiento**

- En streaming se priorizan baja latencia y estabilidad operativa
- En batch se priorizan throughput y coste por volumen procesado
- Separar datos recientes (calientes) de históricos (fríos) ayuda a equilibrar ambos



- **Un buen particionado reparte la carga del pipeline**
 - Nº de particiones en Kafka = paralelismo máximo de consumo
 - En Spark cada partición se procesa como una tarea
 - Si hay pocas particiones el clúster queda infrautilizado
- **Mantiene consistencia en los datos**
 - Agrupa eventos relacionados en la misma partición
 - Conserva el orden dentro de cada partición de Kafka
- **La clave debe tener suficiente granularidad**
 - Claves generales concentran demasiada carga en una partición
 - Clave más granulares mejoran el reparto y el rendimiento
 - Ejemplos son `player_id` o `match_id`



- **Rendimiento es un equilibrio entre throughput, latencia y resiliencia**
 - Depende del tamaño del mensaje, compresión y acks
 - Compresión reduce tráfico de red pero añade coste de CPU
 - acks=all y mayor replicación mejoran durabilidad pero aumentan latencia
- **La infraestructura condiciona el resultado**
 - Kafka es intensivo en entrada/salida → conviene usar discos rápidos
 - La red debe ser estable y con suficiente capacidad para picos de tráfico
 - Es buena práctica separar discos de datos y distribuir réplicas entre racks o zonas



- **Número de particiones**

- Aumentan paralelismo y throughput
- Demasiadas añaden overhead

- **Factores de replicación**

- Aumenta resiliencia
- Aumenta el uso de disco y red

- **Retención y compactación del log**

- Retención de mensajes → definir según su uso
- Compactación del log → mantiene último estado por clave



Replicación en Kafka

Esquema conceptual original de la edición abierta

Fuente: <https://developer.confluent.io/courses/architecture/data-replication/>



- **Batching para agrupar mensajes antes de enviar**
 - Definir tiempo de espera para acumular más mensajes
 - Definir tamaño máximo del batch de mensajes (bytes)
 - Ajustes típicos son `linger.ms` y `batch.size`
 - Más throughput a costa de algo más de latencia de envío
- **Compresión de mensajes para reducir tráfico de red y disco**
 - Distintos algoritmos de compresión con diferentes trade-offs
 - Elegir según el tamaño de los mensajes y la CPU disponible
- **Fiabilidad para no perder eventos críticos**
 - Definir el nivel de confirmación de escritura (`acks`)
 - Activar idempotencia para evitar duplicados por reintentos
 - Más fiabilidad suele implicar más latencia de escritura



- **Consumer groups para escalado horizontal**
 - Añadir consumidores al mismo grupo aumenta el paralelismo
 - El máximo útil es 1 consumidor por partición (extra quedan ociosos)
- **Commits de offset para controlar el progreso**
 - Los commits automáticos simplifican la configuración
 - Los commits manuales dan más control sobre cuándo confirmar
- **Reintentos y DLQs para gestionar errores**
 - Definir una política de reintentos antes de descartar mensajes
 - Enviar a una DLQ los eventos que fallan repetidamente
- **Monitorizar retrasos en consumidores para detectar cuellos de botella**



Coordinación de consumidores

Esquema conceptual original de la edición abierta

Fuente: <https://developer.confluent.io/courses/architecture/consumer-group-protocol/>



- **Particiones, tareas y executors están relacionados**
 - Spark divide los datos en particiones lógicas para procesarlos en paralelo
 - Cada partición suele ejecutarse como una tarea independiente
 - Los executors son los procesos que ejecutan esas tareas en los nodos del clúster
- **El shuffle redistribuye datos entre nodos**
 - Es habitual en operaciones como join, groupBy y agregaciones
 - Suele ser una de las fases más costosas por uso de red, disco y tiempo
- **Las particiones impactan en el rendimiento**
 - Si hay pocas particiones se desaprovecha el clúster
 - Si hay demasiadas aumenta la sobrecarga de gestión
 - Si los executors tienen pocos recursos el procesamiento se ralentiza



- **Las particiones marcan el paralelismo**
 - Cada partición suele ejecutarse como una tarea por etapa
 - Conviene ajustar su número a la capacidad real del clúster
 - El objetivo es equilibrar paralelismo y sobrecarga
- **El desbalanceo de particiones es un enemigo del rendimiento**
 - Aparece cuando pocas particiones concentran muchos datos (skew)
 - Algunas tareas terminan rápido y otras tardan mucho más
 - El rendimiento global queda limitado por la partición más grande
 - Primero se corrige el skew y después se ajusta el número de particiones
- **Decidir entre repartir o compactar particiones**
 - `repartition` → redistribuye datos con *shuffle*
 - `coalesce` → reduce particiones con menos coste (normalmente sin *shuffle*)



- **En skew hay tareas que dominan el tiempo total del job**
 - Pocas tareas lentas dominan el job (stragglers)
 - Uso intensivo de disco mientras otros executors quedan ociosos
- **Es clave comparar la duración de tareas en Spark**
 - Si hay grandes diferencias entre tareas, suele haber desbalanceo
 - Revisar también el tamaño de particiones tras joins y agregaciones
- **Existen diferentes técnicas para mitigar el skew**
 - Repartir mejor la carga (claves más granulares o subclaves)
 - Tratar por separado casos con volumen anómalo
 - Usar AQE para rebalanceo automático



Distribución de datos con skew

Esquema conceptual original de la edición abierta



Distribución equilibrada

Esquema conceptual original de la edición abierta

Fuente: <https://blog.scottlogic.com/2018/03/22/apache-spark-performance.html>



- **El shuffle es la operación más costosa**
 - Implica serialización, red y escritura a disco intermedio
 - Suele dominar el tiempo de ejecución en joins y agregaciones
- **Reducir datos antes del shuffle**
 - Filtrar filas y proyectar solo las columnas necesarias
 - Preagregar cuando sea posible antes de un join
 - Menos datos en movimiento implica menos coste y menor latencia
- **Usar broadcast join con tablas pequeñas**
 - Si una tabla cabe en memoria, se copia a los executors
 - Así se evita redistribuir la tabla grande
 - Genera menos shuffle y joins más rápidos



Operación de shuffle

Esquema conceptual original de la edición abierta

Fuente: <https://blog.scottlogic.com/2018/03/22/apache-spark-performance.html>



- **Guarda datos intermedios para reutilizarlos**
 - Solo compensa cuando esos datos se usan varias veces
 - Si un resultado se usa una sola vez suele empeorar el rendimiento
- **La cache ocupa memoria y compite con la ejecución de tareas**
 - Si no hay memoria suficiente aumenta el uso de disco y baja el rendimiento
 - Conviene priorizar datasets intermedios costosos y estables
- **Buenas prácticas de uso**
 - Cachear después de transformaciones costosas que se reutilizan
 - Liberar el cache cuando deja de ser necesario y validar impacto con métricas



- **Persistir extiende la idea de cache**

- El cache es una forma de persistencia en memoria
- La persistencia permite elegir cómo guardar los datos
- Se usa cuando el mismo DataFrame se reutiliza y el volumen puede ser alto

- **Existen varios niveles de persistencia**

- MEMORY_ONLY → más rápido si los datos caben en memoria
- MEMORY_AND_DISK → más robusto cuando el volumen es grande

- **Checkpointing para análisis en streaming**

- Guarda el estado del pipeline para recuperarse tras fallos
- Es clave en ventanas temporales, deduplicación y joins con estado
- Debe guardarse en almacenamiento fiable (HDFS, S3 o equivalente)



- 1 Introducción
- 2 Buenas prácticas
- 3 Puesta en producción**



- **Elegir el modelo de despliegue**

- Bare metal prioriza rendimiento y control de la infraestructura
- Máquinas virtuales ofrecen más flexibilidad con algo de sobrecarga
- Kubernetes facilita la operación pero exige cuidar el almacenamiento persistente

- **Kafka depende mucho del rendimiento de disco y de red**

- Conviene usar almacenamiento rápido para los logs
- También es importante una red estable entre brokers

- **Es fundamental asegurar la continuidad del servicio**

- Se recomiendan al menos 3 brokers y factor de replicación 3 en tópicos críticos
- Distribuir réplicas en distintos nodos o racks mejora la disponibilidad
- KRaft simplifica el despliegue frente a ZooKeeper

- **Dimensionar almacenamiento y límites de mensajes**



- **Conviene elegir el gestor del clúster según el entorno**
 - YARN encaja bien en ecosistemas Hadoop
 - Kubernetes facilita orquestación, aislamiento y elasticidad
 - Spark standalone suele reservarse para desarrollo y pruebas
- **El almacenamiento condiciona el rendimiento y la operación**
 - HDFS es una opción sólida para cargas batch
 - Checkpointing debe guardarse en almacenamiento fiable
- **Es importante separar cargas y fijar dependencias**
 - Conviene separar recursos de batch y streaming
 - Streaming necesita mayor prioridad para mantener la latencia
 - Docker ayuda a mantener dependencias y versiones estables



- **Conviene elegir entre servicio administrado y control directo**
 - Amazon MSK es Kafka administrado por AWS y reduce trabajo de operación
 - MSK Serverless ajusta recursos automáticamente cuando la carga cambia
 - Kafka en EC2 da más control de configuración pero exige más operación
- **La red y el acceso se controlan dentro de AWS**
- **Seguridad y monitorización deben configurarse desde el inicio**
 - IAM puede gestionar autenticación y autorización de clientes
 - CloudWatch centraliza métricas para monitorizar brokers, tópicos y consumo



- **Conviene elegir la plataforma según el tipo de carga**
 - Amazon EMR encaja bien en batch y en streaming
 - Spark sobre EKS aporta flexibilidad y encaje con contenedores
 - Databricks en AWS simplifica desarrollo y operación
- **En AWS Spark se suele lanzar como jobs**
 - Como EMR Steps, spark-submit en EKS o Jobs en Databricks
- **Es importante diferenciar el uso en streaming y en batch**
 - Un flujo habitual conecta MSK o Kinesis con Spark Structured Streaming
 - En streaming se priorizan latencia y estabilidad del pipeline
 - En batch se priorizan throughput y coste por volumen procesado



- **La monitorización debe centrarse en la salud del pipeline**
 - El retraso del consumidor es la métrica principal para detectar retraso
 - También conviene seguir la tasa de producción y consumo por tópico
 - La latencia de extremo a extremo ayuda a comprobar si se cumplen los objetivos
- **La replicación y el disco son claves para la estabilidad**
 - Problemas en el ISR pueden señalar riesgo de pérdida o menor disponibilidad
 - El uso de disco por broker debe vigilarse de forma continua
- **Las alertas y herramientas deben adaptarse al entorno**
 - Configurar alertas por consumer lag, timeouts y errores de broker
 - En on-premises es habitual usar Prometheus y Grafana
 - En AWS se suele usar CloudWatch y métricas de MSK



Observabilidad de Kafka

Esquema conceptual original de la edición abierta

Fuente: <https://grafana.com/blog/get-comprehensive-monitoring-for-your-apache-kafka-ecosystem-instances-quickly-with-grafana-cloud/>



- **Spark Web UI permite localizar cuellos de botella**
 - En la vista de *stages* se ve qué etapas concentran más tiempo
 - Dentro de cada etapa las *tasks* permiten comparar duraciones y detectar skew
 - Las métricas de *shuffle read/write* muestran cuánto dato se mueve entre nodos
- **En streaming hay que vigilar el ritmo del pipeline**
 - La duración del micro-batch debe ser menor que el intervalo del trigger
 - Comparar datos que entran y datos que se procesan ayuda a ver retrasos
 - Los registros pendientes muestran si se está acumulando trabajo
 - Medir tasa de input y procesamiento, duración del batch y watermark
- **Las alertas deben avisar antes de que el pipeline falle**
 - Alertar si aumenta el uso de disco o el tiempo de limpieza de memoria
 - Alertar de inmediato ante jobs fallidos con contexto del error



Interfaz de monitorización de Spark

Esquema conceptual original de la edición abierta

Fuente: <https://spark.apache.org/docs/latest/web-ui.html>



- El texto, los ejemplos y los esquemas de esta edición abierta se distribuyen bajo CC BY-SA 4.0.
- Las imágenes de procedencia no acreditada de la versión de trabajo se han retirado y sustituido por esquemas conceptuales originales.
- Los logotipos y marcas citados se utilizan únicamente con finalidad identificativa y quedan excluidos de la licencia.
- Las referencias técnicas se conservan mediante citas y enlaces a sus fuentes correspondientes.
- Información de licencia: <https://creativecommons.org/licenses/by-sa/4.0/deed.es>

Big Data Processing II

IA Generativa aplicada al deporte

Prompt Engineering & Post-training

Alberto Fernández Isabel
Natalia Madrueño Sierro
Rubén Rodríguez Fernández

DSLAB

Marzo 2026



Universidad
Rey Juan Carlos



Material docente en abierto de la Universidad Rey Juan Carlos

Asignatura: Big Data Processing II

Titulación: Máster Universitario en Análisis de Datos Deportivos

Autores: Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández

Fecha: 2026

© 2026 Equipo docente. Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional.

Los logotipos, marcas y referencias de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

Depósito institucional: <https://burjcdigital.urjc.es>

Mapa del bloque



Sesión 1

Control del comportamiento

Prompt Engineering + Post-training



Sesión 2

Control de la información

Retrieval-Augmented Generation (RAG)



Sesión 3

Control del flujo y la acción

Agentes, Tools y Arquitecturas Cognitivas

Contexto



- Entender qué es un LLM desde el punto de vista computacional.
- Comprender su papel en la Ciencia de Datos Deportivos.
- Aprender a controlar su comportamiento mediante Prompt Engineering.
- Identificar límites del prompting.



- Modelo probabilístico de generación de texto.
- Aprende patrones lingüísticos.
- Predice la siguiente palabra dada una secuencia.
- Simula coherencia, no conocimiento real.

Arquitectura conceptual de un modelo de lenguaje

Esquema conceptual original de la edición abierta



- **No es una base de datos.**
- **No tiene acceso** a sensores o APIs.
- **No realiza cálculos fiables** por sí mismo.
- **No distingue verdad de plausibilidad.**

Datos deportivos y modelos generativos

Esquema conceptual original de la edición abierta



- Generar narrativas.
- Estandarizar informes.
- Traducir estadísticas a lenguaje humano.
- Interfaz conversacional.

Ciclo de diseño y evaluación

Esquema conceptual original de la edición abierta

Prompt Engineering



Un LLM no se “usa”, se **configura mediante lenguaje**.

- El prompt es el mecanismo con el que definimos:
 - Qué tarea debe realizar.
 - Qué información puede usar.
 - Qué formato debe producir.
- En un sistema de datos, el prompt actúa como:
 - Capa de control sobre el comportamiento del modelo.
 - Sustituto de una interfaz de programación tradicional.
- El modelo no cambia internamente → solo cambia su comportamiento en esa interacción.

Idea clave: Prompt Engineering = programación en lenguaje natural.



Un error común es tratar al LLM como si fuera un buscador o una persona.

- **Pregunta informal:**

“Resume el partido de ayer.”

- Ambigua.
- Sin contexto.
- Sin criterio de calidad.

- **Especificación de tarea:**

*“Genera un resumen técnico de máximo 5 líneas usando SOLO las métricas dadas.
Estructura: contexto, acciones clave, impacto táctico.”*

Diferencia fundamental: Las preguntas producen texto. Las especificaciones producen resultados utilizables en un sistema.



Un prompt profesional no es una frase, es una estructura:

- **Rol (Role)** Define el perfil desde el que debe responder el modelo. Ej.: “Eres analista de rendimiento en un club profesional”.
- **Tarea (Task)** Especifica exactamente qué debe hacer. Ej.: “Genera un informe técnico del jugador X”.
- **Contexto (Context)** Información disponible para realizar la tarea. Ej.: estadísticas, partido, periodo, posición del jugador.
- **Restricciones (Constraints)** Limita el comportamiento del modelo. Ej.: “No inventes datos”, “Usa solo las métricas dadas”.
- **Formato de salida (Output Format)** Cómo debe entregarse la respuesta. Ej.: JSON, tabla, bullets, informe estructurado.

Idea clave: un buen prompt reduce grados de libertad del modelo.



Prompt: *“Analiza el rendimiento del jugador.”*

- **Ambigüedad de objetivo:** ¿rendimiento ofensivo, defensivo, físico, táctico?
- **Sin contexto:** no hay periodo (partido, mes, temporada) ni rival ni rol.
- **Sin datos:** el modelo tenderá a **inventar** métricas o asumirlas.
- **Sin criterio:** no defines qué significa “bueno/malo” (percentiles, umbrales).
- **Sin formato:** imposible integrar la salida en un pipeline (ETL/dashboard).

Diagnóstico Big Data: prompt mal definido = salida no reutilizable + riesgo alto de alucinación.



Prompt (plantilla):

- **Rol:** “Eres analista de rendimiento en un club profesional”.
- **Tarea:** “Genera un informe técnico del jugador X”.
- **Contexto:** “Usa SOLO las métricas proporcionadas (tabla adjunta)”.
- **Restricciones:** “No inventes datos; si falta una métrica, marca null”.
- **Formato:** “Devuelve JSON con: resumen, puntos fuertes, puntos débiles, alertas”.

Efecto: reduces grados de libertad y conviertes la interacción en una “especificación”.



Entrada (ejemplo): métricas por 90 minutos (últimos 5 partidos)

- minutes=432, goals=2, assists=1, xG=1.6, xA=0.9
- shots_p90=2.8, key_passes_p90=1.7, passes_acc=84%
- pressures_p90=14.2, tackles_p90=1.1, interceptions_p90=0.8
- distance_km=10.9, sprints=21, high_intensity_runs=52

Nota: esto es intencionalmente “poco” para forzar que el prompt gestione faltantes sin inventar.



Objetivo: que la salida sea consumible por procesos automáticos (API/ETL/dashboard)

```
{
  "player": "X",
  "period": "last_5_matches",
  "summary": "... (3-5 lineas)",
  "strengths": ["...", "..."],
  "weaknesses": ["...", "..."],
  "alerts": [{"type": "fatigue_risk", "evidence": "..."}],
  "metrics_used": ["minutes", "xG", "xA", "pressures_p90"],
  "missing_metrics": ["duels_won_pct", "progressive_passes_p90"]
}
```

Clave: el modelo no “se luce”; produce un contrato de datos.



- **Trazabilidad:** puedes auditar qué campos rellena y cuáles faltan.
- **Evaluación automática:** validación de esquema (JSON schema), checks.
- **Integración:** dashboard/BI, storage, pipelines batch/stream.
- **Reproducibilidad:** mismo prompt + mismos datos → salida comparable.
- **Mitigación de alucinación:** forzar null/unknown evita inventar.

Regla de oro: si lo vas a usar en un sistema, **no pidas texto libre**.



- Un prompt operativo funciona como un **contrato** entre:
 - **Entrada** (datos disponibles) y **Salida** (estructura/criterios).
 - **Responsabilidades** del modelo (qué puede inferir vs qué no).
- En un proyecto real debes:
 - **Versionar** prompts (v1, v1.1...) igual que código.
 - Registrar: modelo, temperatura, prompt, y cambios de esquema.
 - Mantener tests con ejemplos (golden set) para evitar regresiones.

Puente a técnicas: para consistencia, entra Zero/One/Few-shot.



- Un prompt largo no es “mejor”: puede introducir contradicciones o ruido.
- Buen prompt = **instrucciones cortas + estructura + ejemplos mínimos**.
- Coste típico en sistemas: tokens (entrada + salida) $\Rightarrow$ dinero + latencia.
- Estrategia práctica:
 - Plantillas reutilizables.
 - Contexto **solo** necesario.
 - Ejemplos (few-shot) compactos.
 - Formato fijo para minimizar longitud de salida.

Mensaje para el máster: Prompt engineering también es **optimización de sistema**.

Técnicas de prompting



- El modelo resuelve la tarea solo con instrucciones.
- Se apoya en su conocimiento general aprendido en pretraining.
- Ventajas:
 - Bajo coste en tokens.
 - Rápido de diseñar.
 - Flexible ante tareas variadas.
- Limitaciones:
 - Menor consistencia de formato.
 - Más variabilidad en la respuesta.



Tarea: Clasificar tipo de evento táctico.

Prompt:

“Clasifica el siguiente evento como: transición ofensiva, ataque posicional o contraataque: ‘Recuperación en campo propio, pase largo al espacio, finalización en 8 segundos.’”

Observación: funciona, pero puede variar la explicación y formato.



- Se añade un ejemplo de entrada–salida correcta.
- El modelo imita patrón, estructura y nivel de detalle.
- Útil para:
 - Informes de scouting.
 - Clasificaciones consistentes.
 - Resúmenes con plantilla fija.



Ejemplo dado al modelo:

Entrada: métricas de un jugador.

Salida: informe estructurado con fortalezas, debilidades y alerta física.

Luego se pide lo mismo para otro jugador.

Efecto: se estandariza la estructura sin entrenar el modelo.



- Varios ejemplos refuerzan el patrón deseado.
- Reduce ambigüedad semántica.
- Mejora estabilidad del formato.
- Especialmente útil cuando:
 - hay terminología técnica
 - la tarea es subjetiva (evaluaciones)

Aprendizaje mediante ejemplos en contexto

Esquema conceptual original de la edición abierta



Comparación zero-shot, one-shot y few-shot

Esquema conceptual original de la edición abierta



- Obliga al modelo a generar pasos intermedios.
- Aumenta probabilidad de razonamientos coherentes.
- Reduce errores en tareas lógicas y causales.
- No es razonamiento real, es secuencialización probabilística.

Descomposición explícita de una tarea

Esquema conceptual original de la edición abierta



Pregunta: ¿Por qué bajó el rendimiento ofensivo?

Modelo debe razonar:

- Reducción de minutos.
- Menos acciones en último tercio.
- Aumento de carga física.



CoT es útil cuando la tarea requiere descomposición lógica o causal:

- **Diagnóstico de rendimiento deportivo** Identificar causas de variaciones en métricas (minutos, carga, acciones).
- **Evaluación táctica** Analizar secuencias de juego paso a paso para inferir decisiones tácticas.
- **Explicaciones causales** Relacionar eventos: fatiga → menor intensidad → menos acciones decisivas.
- **Razonamiento multi-variable** Cuando intervienen estadísticas físicas, técnicas y contextuales.

Efecto: fuerza al modelo a estructurar el proceso antes de la conclusión.



Aunque mejora la coherencia, CoT introduce efectos adversos importantes:

- **Aumento del coste computacional** Generar pasos intermedios incrementa tokens de salida → mayor latencia y coste en sistemas en producción.
- **Ilusión de racionalidad** El modelo puede producir razonamientos estructurados pero incorrectos, dando una falsa sensación de validez.
- **Propagación de errores** Un fallo en un paso intermedio contamina el resto del razonamiento.
- **Mayor exposición a alucinaciones** Cuanto más texto genera el modelo, más oportunidades tiene de introducir información no fundamentada.
- **No aporta verificación factual** CoT mejora forma, no veracidad. No sustituye RAG ni validación externa.

Conclusión: CoT mejora estructura del razonamiento, pero no garantiza corrección.

Errores comunes en Prompt Engineering



La ambigüedad introduce variabilidad no controlada en la salida del modelo.

- No se define con precisión la **tarea objetivo**.
- El modelo debe inferir intención → introduce suposiciones.
- Aumenta la **varianza entre ejecuciones**.
- Complica la evaluación automática y la comparación de resultados.

Impacto en sistemas: salida impredecible → difícil integrar en pipelines. **Ejemplo:** “Analiza el partido” → ¿rendimiento físico, táctica, psicología?



El modelo no detecta “lagunas”; las rellena probabilísticamente.

- Si faltan datos, el modelo usa conocimiento general.
- Aumenta riesgo de **alucinaciones plausibles**.
- Se diluye la frontera entre información real e inferida.
- En deporte: puede inventar estadísticas, roles o situaciones.

Regla operativa: lo que no está en el prompt no forma parte de la realidad del sistema.



Sin formato definido, la salida es texto narrativo no estructurado.

- No se puede validar automáticamente.
- Difícil de almacenar y consultar.
- No integrable en pipelines Big Data.
- Alta variabilidad de estructura entre ejecuciones.

Impacto técnico: se rompe la trazabilidad del sistema. **Solución:** definir siempre esquema de salida (JSON, tabla, claves fijas).



Un prompt multiobjetivo diluye el foco del modelo.

- Clasificar + explicar + evaluar en un solo prompt.
- El modelo reparte capacidad entre subtareas.
- Respuestas más superficiales y menos precisas.
- Mayor probabilidad de incoherencias internas.

Principio de diseño: un prompt → una tarea principal.



La coherencia lingüística no implica veracidad.

- Los LLMs son modelos probabilísticos, no sistemas de verificación.
- Pueden producir respuestas seguras pero incorrectas.
- Mayor riesgo cuando la respuesta parece bien argumentada.
- En deporte: decisiones basadas en datos inventados pueden ser críticas.

Requisito de sistema: validación humana o cruzada con fuentes reales.



Un prompt no evaluado es un componente no controlado del sistema.

- Sin métricas no se puede optimizar ni comparar versiones.
- Los cambios en prompts afectan al comportamiento global del sistema.
- La calidad percibida puede no reflejar estabilidad real.

Resultado: deriva del sistema sin que se detecte.



Un prompt debe evaluarse como cualquier otro componente software.

- Consistencia del formato (JSON válido, claves presentes).
- Tasa de campos vacíos o incorrectos.
- Variabilidad entre ejecuciones con la misma entrada.
- Coherencia con datos reales.
- Estabilidad tras cambios de modelo o temperatura.

Objetivo: minimizar varianza y maximizar fiabilidad operativa.

Post-training de LLMs



- Controla formato y tarea.
- Reduce ambigüedad.
- Mejora consistencia.
- Pero no cambia lo que el modelo sabe ni cómo está alineado.

Por eso existe el post-training.



Un modelo tras el pretraining:

- Ha aprendido a predecir texto.
- Conoce patrones lingüísticos.
- Tiene conocimiento general.

Pero:

- No está optimizado para seguir instrucciones.
- No está alineado con expectativas humanas.
- No está diseñado para ser útil como asistente.



- Conjunto de técnicas que se aplican tras el entrenamiento base.
- Objetivo: modificar el **comportamiento** del modelo.
- No enseña nuevo lenguaje, enseña **cómo responder**.

Idea clave: transforma un generador de texto en un asistente usable.



Ciclo de post-entrenamiento

Esquema conceptual original de la edición abierta



- Que el modelo siga instrucciones.
- Mejorar claridad y estructura de respuestas.
- Controlar tono y estilo.
- Reducir respuestas peligrosas o inapropiadas.
- Enseñar cuándo abstenerse de responder.



- Ajuste con ejemplos humanos correctos.
- El modelo imita respuestas bien estructuradas.
- Introduce estilo conversacional.

Limitación: solo aprende por imitación, no por preferencias.



- Variante de SFT centrada en seguir instrucciones.
- Se entrena con muchas tareas distintas.
- Mejora versatilidad del modelo.

Resultado: menos necesidad de prompts complejos.



- Reinforcement Learning from Human Feedback.
- Humanos comparan respuestas del modelo.
- El modelo aprende qué respuestas son preferidas.

Efecto: mayor naturalidad y alineación con expectativas humanas.



- RLAIIF: feedback generado por modelos IA.
- DPO: optimización directa por preferencias.
- Más simples y escalables que RLHF clásico.



- Adaptación a áreas específicas (medicina, derecho, deporte).
- Mejora precisión terminológica.
- Respuestas más especializadas.

Riesgo: pérdida de generalidad y sesgos de dominio.



- El modelo:
 - Sigue mejor instrucciones.
 - Es más conversacional.
 - Es más prudente.
- Pero:
 - Sigue siendo probabilístico.
 - Sigue alucinando.
 - No tiene acceso a datos externos.



- Prompting = control externo.
- Post-training = comportamiento interno.

Metáfora:

- Post-training educa al modelo.
- Prompting le da instrucciones concretas.



- Alineación imperfecta.
- Alucinaciones persistentes.
- Puede parecer seguro sin ser correcto.
- No sustituye acceso a datos reales.



- Modelos demasiado conservadores.
- Respuestas genéricas y seguras.
- Sesgos heredados del feedback humano.
- Puede priorizar parecer correcto sobre ser útil.



- Un LLM no es una fuente de conocimiento.
- El prompt es una herramienta de ingeniería.
- Las técnicas de prompting mejoran control, no verdad.
- El comportamiento base del modelo viene del post-training.
- Para trabajar con datos reales necesitaremos RAG.



- El texto, los ejemplos y los esquemas de esta edición abierta se distribuyen bajo CC BY-SA 4.0.
- Las imágenes de procedencia no acreditada de la versión de trabajo se han retirado y sustituido por esquemas conceptuales originales.
- Los logotipos y marcas citados se utilizan únicamente con finalidad identificativa y quedan excluidos de la licencia.
- Las referencias técnicas se conservan mediante citas y enlaces a sus fuentes correspondientes.
- Información de licencia: <https://creativecommons.org/licenses/by-sa/4.0/deed.es>

Big Data Processing II

IA Generativa aplicada al deporte

Retrieval-Augmented Generation (RAG)

Alberto Fernández Isabel
Natalia Madrueño Sierro
Rubén Rodríguez Fernández

DSLAB

Marzo 2026



Universidad
Rey Juan Carlos



Material docente en abierto de la Universidad Rey Juan Carlos

Asignatura: Big Data Processing II

Titulación: Máster Universitario en Análisis de Datos Deportivos

Autores: Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández

Fecha: 2026

© 2026 Equipo docente. Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional.

Los logotipos, marcas y referencias de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

Depósito institucional: <https://burjcdigital.urjc.es>

Mapa del bloque



Sesión 1

Control del comportamiento

Prompt Engineering + Post-training



Sesión 2

Control de la información

Retrieval-Augmented Generation (RAG)



Sesión 3

Control del flujo y la acción

Agentes, Tools y Arquitecturas Cognitivas

Introducción



Un LLM modela una distribución condicional:

$$P(y \mid x)$$

donde:

- x = prompt + contexto disponible
- y = secuencia generada

Pero:

- No tiene acceso directo a bases de datos externas.
- No consulta sistemas internos.
- No distingue entre dato real y patrón aprendido.

Conclusión: conocimiento aprendido $\neq$ acceso a información actualizada.



El Prompt Engineering:

- Controla estructura.
- Reduce ambigüedad.
- Define formato.

Pero:

Prompting $\nrightarrow$ Acceso a datos externos

No modifica la base informacional del modelo.

Arquitectura general de recuperación aumentada

Esquema conceptual original de la edición abierta



Un sistema real dispone de:

- Tracking físico por partido.
- Métricas avanzadas (xG, PPDA, presiones).
- Informes médicos.
- Observaciones de scouting.

Pregunta crítica:

¿Ha disminuido la intensidad de presión tras acumular más de 300 minutos en 10 días?

Un LLM aislado no puede responder correctamente.

Definición de RAG



Retrieval-Augmented Generation introduce un mecanismo externo:

$$Output = f(Prompt + Retrieve(Query))$$

- Retrieve(Query) devuelve fragmentos relevantes.
- El LLM genera condicionado por ese contexto.

No cambia parámetros del modelo. Cambia la evidencia disponible en inferencia.



Componente	Rol
Embeddings	Representación semántica
Vector DB	Indexación eficiente
Retriever	Selección de contexto
LLM	Síntesis condicionada

RAG es una arquitectura distribuida, no una simple extensión del modelo.

Flujo de recuperación y generación

Esquema conceptual original de la edición abierta

Diseño técnico de RAG



Dividir documentos implica:

- Elegir **tamaño de fragmento**.
- Definir **solapamiento**.
- Preservar **coherencia semántica**.

Errores comunes:

- Chunk demasiado grande → baja precisión.
- Chunk demasiado pequeño → pérdida de contexto.
- Solapamiento excesivo → redundancia y coste.

Estrategias de fragmentación

Esquema conceptual original de la edición abierta



Un embedding es una función:

$$E : \text{Texto} \rightarrow \mathbb{R}^d$$

Permite medir similitud semántica.

Limitaciones:

- No captura bien relaciones numéricas exactas.
- Puede perder contexto temporal.
- Similaridad $\neq$ relevancia estadística.



Similitud típica:

$$\text{sim}(u, v) = \frac{u \cdot v}{\|u\| \|v\|}$$

Pero:

- Similaridad semántica no garantiza pertinencia.
- Sensible a redacción del query.

La calidad del retrieval es crítica.

Representación vectorial

Esquema conceptual original de la edición abierta



Un sistema robusto debe permitir:

- Filtrado por temporada, jugador, competición.
- Indexación por tipo de documento.
- Actualización incremental.

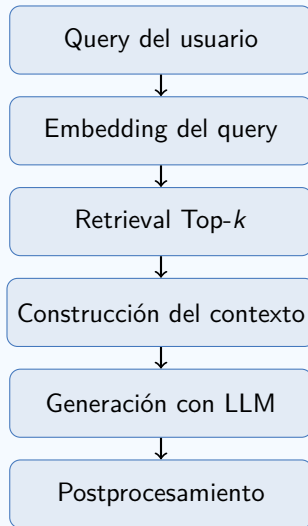
Sin metadatos, el retrieval pierde precisión contextual.

Caso de recuperación aplicado al fútbol

Esquema conceptual original de la edición abierta



Cada etapa introduce posible error.





Veracidad $\uparrow \Rightarrow$ *Coste* $\uparrow$

- Más contexto $\rightarrow$ más tokens.
- Más tokens $\rightarrow$ mayor latencia.
- Mayor latencia $\rightarrow$ menor escalabilidad.

Diseñar **RAG** es **optimizar** este equilibrio.

Compromisos entre recuperación y generación

Esquema conceptual original de la edición abierta

Aplicaciones en deporte



Base interna:

- Métricas por partido.
- Indicadores avanzados.
- Carga física acumulada.

Consulta:

Compara evolución de presión alta con descanso < 72h.

Requiere filtrado + recuperación + síntesis.

Pipeline RAG

Esquema conceptual original de la edición abierta



Documentos:

- Informes técnicos.
- Observaciones cualitativas.

Permite:

- Búsqueda semántica.
- Detección de patrones recurrentes.
- Síntesis multi-documento.

Ejemplo de consulta con contexto deportivo

Esquema conceptual original de la edición abierta



Problema:

Embeddings no gestionan bien números exactos.

Solución:

- **Recuperación textual** (búsqueda semántica en RAG).
- **Cálculo determinista externo** (fórmulas programadas).
- **Inyección de resultados estructurados** (creación de tablas numéricas).

Arquitectura híbrida = mayor precisión.

Limitaciones



RAG introduce dos fuentes de error:

$$Error_{total} = Error_{retrieval} + Error_{generation}$$

Un mal retrieval no se compensa con mejor modelo.

Fuentes de error en RAG

Esquema conceptual original de la edición abierta



Si el documento contiene error:

- **El modelo lo amplifica.**
- **No detecta contradicciones internas.**
- **No valida consistencia factual.**

RAG mejora acceso, no garantiza verdad.

Evaluación



- Precision@k
- Recall@k
- Mean Reciprocal Rank (MRR)
- nDCG

Sin evaluar retrieval, no se puede optimizar el sistema.

Evaluación del flujo RAG

Esquema conceptual original de la edición abierta



Se debe medir:

- **Coherencia final.**
- **Consistencia con fuente.**
- **Estabilidad entre ejecuciones.**
- **Trazabilidad** (citar fragmentos).

RAG sin trazabilidad es **opaco** y sin estabilidad **no es confiable**.

Conclusiones



En datos dinámicos, RAG suele ser preferible.

Fine-tuning	RAG
Cambia parámetros	No cambia parámetros
Integra conocimiento	Consulta conocimiento
Coste alto inicial	Coste por consulta
Riesgo de olvido	Modelo intacto



Es una arquitectura de **acceso a información**, no un mecanismo de aprendizaje.

- RAG no hace al modelo más inteligente.
- El sistema se especializa en un contexto.
- Incrementa el tamaño del prompt.
- Afecta al número de tokens utilizados.

Siguiente paso: arquitecturas agénticas.



- El texto, los ejemplos y los esquemas de esta edición abierta se distribuyen bajo CC BY-SA 4.0.
- Las imágenes de procedencia no acreditada de la versión de trabajo se han retirado y sustituido por esquemas conceptuales originales.
- Los logotipos y marcas citados se utilizan únicamente con finalidad identificativa y quedan excluidos de la licencia.
- Las referencias técnicas se conservan mediante citas y enlaces a sus fuentes correspondientes.
- Información de licencia: <https://creativecommons.org/licenses/by-sa/4.0/deed.es>

Big Data Processing II

IA Generativa aplicada al deporte

Sistemas basados en agentes

Alberto Fernández Isabel
Natalia Madrueño Sierro
Rubén Rodríguez Fernández

DSLAB

Marzo 2026



Universidad
Rey Juan Carlos



Material docente en abierto de la Universidad Rey Juan Carlos

Asignatura: Big Data Processing II

Titulación: Máster Universitario en Análisis de Datos Deportivos

Autores: Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández

Fecha: 2026

© 2026 Equipo docente. Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional.

Los logotipos, marcas y referencias de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

Depósito institucional: <https://burjcdigital.urjc.es>

Mapa del bloque



Sesión 1

Control del comportamiento

Prompt Engineering + Post-training



Sesión 2

Control de la información

Retrieval-Augmented Generation (RAG)



Sesión 3

Control del flujo y la acción

Agentes, Tools y Arquitecturas Cognitivas

Introducción



Un LLM tradicional:

$$\text{Output} = f(\text{prompt})$$

Un agente:

$$\text{acción}_{t+1} = f(\text{estado}_t, \text{objetivo}, \text{observación}_t)$$

- No solo responde.
- Decide.
- Actúa.
- Aprende del entorno.

Arquitectura de un agente

Esquema conceptual original de la edición abierta



LLM + RAG permiten:

- Responder preguntas.
- Sintetizar información.

Pero no permiten:

- Ejecutar **cálculos complejos**.
- **Consultar** múltiples sistemas.
- **Tomar decisiones** secuenciales.
- **Adaptarse dinámicamente**.

Integración de RAG y agentes

Esquema conceptual original de la edición abierta

Skills y tools



Un agente se compone de **habilidades (skills)**:

- consultar bases de datos
- ejecutar modelos estadísticos
- calcular métricas avanzadas
- generar visualizaciones
- enviar alertas automáticas

Un skill es una **capacidad reutilizable**.

Relación entre agentes, skills y tools

Esquema conceptual original de la edición abierta



Tool	Skill
Función aislada	Capacidad compuesta
Ejecuta acción	Orquesta acciones
Ej: query DB	Ej: análisis rendimiento



Ejemplos en deporte:

- API de tracking físico.
- Motor de cálculo de xG (goles esperados).
- Base histórica de rendimiento.
- Generador de dashboards.

El valor del agente crece con su ecosistema.

Agente de inteligencia artificial

Esquema conceptual original de la edición abierta

Planificación y razonamiento iterativo



Los agentes complejos no actúan en un solo paso.

Proceso:

- 1 **Analizar objetivo.**
- 2 **Descomponer subtareas.**
- 3 **Ejecutar pasos.**
- 4 **Evaluar resultados.**
- 5 **Continuar o finalizar.**

Flujo de decisión agéntico

Esquema conceptual original de la edición abierta



Arquitectura **ReAct**:

- **Pensamiento** intermedio.
- **Selección** de acción.
- **Ejecución** de tool.
- **Observación** del resultado.
- **Iteración**.

Combina **razonamiento y acción**.

Patrón ReAct

Esquema conceptual original de la edición abierta



Arquitectura **moderna**:

- **Planner**: diseña los pasos
- **Executor**: ejecuta herramientas
- **Evaluator**: valida resultados

Permite tareas complejas multi-etapa.

Orquestación de agentes

Esquema conceptual original de la edición abierta

Arquitecturas dirigidas por estado (control de flujo)



Los flujos lineales (chains) fallan cuando:

- La ruta depende del contenido (distintos casos de uso).
- Hay pasos opcionales (solo si faltan datos, solo si hay anomalía).
- Hay iteración (refinar consulta, reintentar retrieval, validar).
- Se necesitan **invariantes** (no continuar si el JSON no valida).

Necesidad: control explícito del flujo y de las condiciones de parada.



Una forma rigurosa de diseñar agentes es modelarlos como:

$$S \xrightarrow{\text{evento/condición}} S'$$

- **Estados** = etapas del proceso (retrieve, compute, write-report, validate).
- **Transiciones** = reglas (si falta dato → consultar DB; si inconsistente → reintentar).
- **Guardas** = condiciones que evitan errores (no generar si no hay evidencias).

Ventaja: control del sistema > “improvisación del LLM”.

Estado y memoria

Esquema conceptual original de la edición abierta



Patrones típicos en agentes:

- **Router**: elegir herramienta según intención (consulta histórica vs cálculo).
- **Validator**: comprobar esquema/consistencia antes de seguir.
- **Retry with backoff**: reintentos controlados ante fallos de tool/timeout.
- **Loop bounded**: iteración con límite de pasos para evitar loops infinitos.

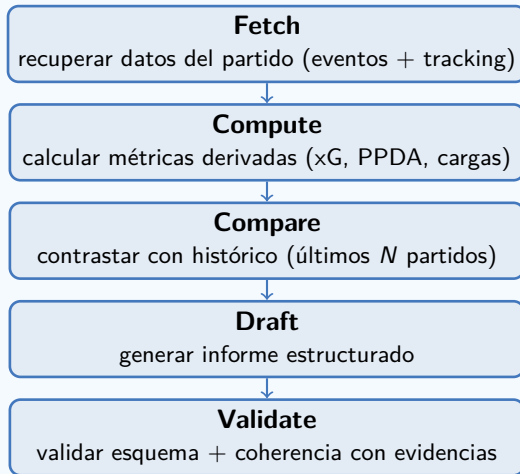
Idea clave: la robustez viene del control de flujo, no del “prompt perfecto”.

Patrones de arquitectura agéntica

Esquema conceptual original de la edición abierta



Estados de alto nivel:



Routing: si faltan métricas → volver a Fetch; si el JSON falla → volver a Draft.



- **Loops infinitos:** imponer límite de iteraciones y condiciones de parada.
- **Tool misuse:** validación de argumentos + esquemas estrictos.
- **Saltos incorrectos:** transiciones solo si se cumplen guardas.
- **Acumulación de latencia:** cortar ramas no necesarias y cachear resultados.

Conclusión: el estado gobierna el flujo; el LLM decide dentro de límites.

Memoria y estado



Estado:

- Representación del **proceso actual** (qué sabemos ahora y qué falta).
- Vive dentro de la ejecución (por ejemplo, un diccionario/JSON de trabajo).

Memoria:

- Persistencia entre ejecuciones: historial, preferencias, perfiles, conocimiento externo.
- Vive fuera del flujo principal (DB, vector store, logs, cache).

Problema habitual: mezclar ambos genera agentes incoherentes y difíciles de depurar.



- **Conversacional (corto plazo):** historial reciente (context window).
- **Episódica:** interacciones previas resumidas (resúmenes por sesión).
- **Semántica:** documentos y conocimiento recuperable (RAG).
- **Estructurada:** perfiles, preferencias, configuraciones (tablas/JSON persistente).

Idea clave: cada tipo tiene coste, latencia y riesgos distintos.

Razonamiento y planificación

Esquema conceptual original de la edición abierta



Preguntas que deben tener respuesta:

- **Qué se guarda** (señal vs ruido).
- **Cuándo se guarda** (al final de tarea, tras validación).
- **Cómo se recupera** (por claves, por embeddings, por filtros).
- **Caducidad** (TTL): lo viejo puede perjudicar.
- **Auditoría**: logs trazables para explicar decisiones.

Objetivo: memoria útil, no acumulación de texto.

Modelo cognitivo de un agente

Esquema conceptual original de la edición abierta



Memoria estructurada: perfil del jugador (posición, rol, minutos, lesiones).

Memoria semántica: informes textuales de scouting y clips anotados.

Memoria episódica: decisiones previas del agente (por qué recomendó A vs B).

Consulta:

“Dame 3 perfiles similares a este extremo, justificando con evidencias de informes.”

Resultado: respuestas consistentes porque el agente recuerda y cita.



- **Contaminación** (guardar alucinaciones)
→ guardar solo tras validación.
- **Obsolescencia** → TTL y versionado (temporada, fecha, fuente).
- **Privacidad/seguridad** → control de acceso + minimización de datos.
- **Sesgo por historiales** → resumir y normalizar, no copiar todo.

Conclusión: la memoria aumenta capacidad, pero sin control degrada fiabilidad.

Riesgos de memoria persistente

Esquema conceptual original de la edición abierta

Riesgos y fallos reales en agentes



A mayor autonomía del agente:

- mayor número de componentes,
- mayor superficie de fallo,
- mayor dificultad de depuración.

Los fallos rara vez provienen del modelo aislado.

Conclusión: la memoria aumenta capacidad, pero sin control degrada fiabilidad.

Complejidad de un sistema agéntico

Esquema conceptual original de la edición abierta



Problemas frecuentes:

- iteraciones sin condición de parada,
- reintentos infinitos de retrieval,
- planificación redundante.

Mitigación:

- límites de iteración,
- condiciones de parada explícitas,
- control del estado.

Conclusión: la memoria aumenta capacidad, pero sin control degrada fiabilidad.

Bucle de percepción y acción

Esquema conceptual original de la edición abierta



El agente puede:

- invocar tools equivocadas,
- usar parámetros inválidos,
- interpretar mal los resultados.

Mitigación:

- esquemas de entrada estrictos,
- validación automática,
- manejo robusto de errores.

Herramientas externas

Esquema conceptual original de la edición abierta



Riesgo crítico:

- el sistema produce resultados plausibles pero incorrectos.

Soluciones:

- Logging detallado,
- Trazabilidad de evidencias,
- Auditoría de decisiones.

Sin trazabilidad, el sistema no es confiable.

Coordinación multiagente

Esquema conceptual original de la edición abierta



Los agentes pueden:

- ejecutar múltiples herramientas,
- generar grandes cantidades de tokens,
- realizar múltiples iteraciones.

Resultado:

- latencias elevadas,
- costes operativos altos.

Optimizar flujo es esencial.

Control de costes

Esquema conceptual original de la edición abierta



Si los datos son incorrectos:

- el agente amplifica errores,
- las conclusiones pueden ser erróneas,
- aumenta el riesgo en decisiones deportivas.

Regla: garbage in → garbage out.

Evaluación de calidad

Esquema conceptual original de la edición abierta



- Validar entradas y salidas.
- Limitar iteraciones y costes.
- Registrar decisiones y evidencias.
- Diseñar flujos controlados por estado.
- Mantener supervisión humana.

Caso deportivo de ejemplo

Esquema conceptual original de la edición abierta

Conclusión: robustez > autonomía.

Conclusiones



1. Un agente no es un LLM con más prompts

- Es un sistema que percibe, decide y actúa.
- El LLM es el motor de decisión, no el sistema completo.

2. El valor real surge de integrar herramientas y datos

- Los agentes delegan cálculos y consultas a tools externas.
- La precisión proviene de combinar IA generativa y procesos deterministas.

3. El control del flujo y del estado determina la robustez

- Arquitecturas dirigidas por estado evitan comportamientos erráticos.
- Sin control del flujo aparecen loops, errores y costes excesivos.

4. La memoria y la trazabilidad permiten sistemas fiables

- Registrar evidencias y decisiones es esencial para confiar en el sistema.

Conclusión: los agentes no añaden “inteligencia mágica”; permiten construir sistemas que toman decisiones útiles en entornos reales.



- El texto, los ejemplos y los esquemas de esta edición abierta se distribuyen bajo CC BY-SA 4.0.
- Las imágenes de procedencia no acreditada de la versión de trabajo se han retirado y sustituido por esquemas conceptuales originales.
- Los logotipos y marcas citados se utilizan únicamente con finalidad identificativa y quedan excluidos de la licencia.
- Las referencias técnicas se conservan mediante citas y enlaces a sus fuentes correspondientes.
- Información de licencia: <https://creativecommons.org/licenses/by-sa/4.0/deed.es>

Big Data Processing II

IA Generativa aplicada al deporte

Tema 5: Proyectos integrados

Del dato en tiempo real al análisis automatizado

Alberto Fernández Isabel
Natalia Madrueño Sierro
Rubén Rodríguez Fernández

DSLAB

Abril 2026





Material docente en abierto de la Universidad Rey Juan Carlos

Asignatura: Big Data Processing II

Titulación: Máster Universitario en Análisis de Datos Deportivos

Autores: Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández

Fecha: 2026

© 2026 Equipo docente. Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional.

Los logotipos, marcas y referencias de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

Depósito institucional: <https://burjcdigital.urjc.es>



- Comprender la transición del análisis retrospectivo (BI tradicional) a la toma de decisiones prescriptiva en tiempo real.
- Diseñar infraestructuras integradas combinando ingestión masiva (Kafka) y cómputo continuo (Spark).
- Desplegar sistemas de IA generativa y ecosistemas multiagente sobre flujos de datos deportivos.
- Implementar arquitecturas RAG multimodales para la comprensión espacio-temporal del juego.
- Automatizar la generación de informes y narrativas adaptadas a diferentes perfiles de stakeholders.



- 1 Introducción: El paradigma del análisis automatizado
- 2 Arquitecturas completas de ingestión y procesamiento
- 3 Orquestación de IA Generativa y flujos agénticos
- 4 RAG Multimodal y Generación de Narrativas
- 5 Visualización avanzada e interfaces conversacionales
- 6 Conclusiones

Introducción al paradigma automatizado



- La ciencia de datos deportivos ya no puede depender exclusivamente de los informes post-partido.
- El valor del dato decae exponencialmente desde el momento en que ocurre la acción.
- **Objetivo moderno:** Proveer a los entrenadores y analistas de herramientas que sugieran ajustes tácticos durante el desarrollo de la competición.
- Se exige una transición arquitectónica hacia la analítica predictiva y automatizada.



La infraestructura actual requiere la convergencia de tres pilares:

- **Ingestión continua:** Captura ininterrumpida de biometría, GPS y vídeo.
- **Motor determinista:** Cálculos estadísticos en ventanas temporales.
- **Capa cognitiva (IA):** Modelos de lenguaje que interpretan y traducen matemáticas en táctica.

Esta convergencia permite pasar de paneles de control estáticos a "**sistemas observacionales**" inteligentes.



- Un LLM tradicional es una "foto fija" probabilística basada en su entrenamiento.
- **El problema en el deporte:** Carece de contexto del estado actual del mundo; no sabe quién está fatigado ni cuál es el marcador en el minuto 75.
- **Solución:** Integrar al LLM como un motor de razonamiento (*reasoning engine*) dentro de un pipeline de datos alimentado en vivo.



- La automatización total abarca desde la captura del evento micro-táctico hasta la narrativa macroscópica.
- Requiere superar los límites epistemológicos de los enfoques batch.
- El contexto inmediato permite extraer valor competitivo en la ventana crítica de la toma de decisiones.



- ❶ **Fuentes (Productores):** Tracking óptico, sensores de balones, wearables.
- ❷ **Middleware de Mensajería:** Enrutamiento masivo y ordenado.
- ❸ **Motor de Streaming:** Agregación e identificación de anomalías.
- ❹ **Agentes LLM / RAG:** Generación de diagnósticos e informes.
- ❺ **Destinos (Consumidores):** Aplicaciones de cuerpo técnico, retransmisiones (broadcasters).

Arquitecturas completas de ingestión



- Entornos de alta presión (estadios repletos) con conectividad variable.
- **Teorema CAP aplicado al deporte:**
 - Consistencia (C), Disponibilidad (A), Tolerancia a Particiones (P).
- En un partido en directo, la latencia es crítica.
- Generalmente, se prioriza la **Disponibilidad** y la **Tolerancia a fallos** de red (AP) sobre la consistencia matemática perfecta que llegaría tarde.



- Apache Kafka actúa como el núcleo de mensajería (Pub/Sub).
- **Desacoplamiento:** Los sensores (visión, GPS) no saben quién consume los datos.
- **Ventaja:** Permite que múltiples sistemas (motores de cálculo de xG, sistemas de IA generativa para alertas tácticas y bases de datos históricas) consuman el mismo evento de forma independiente y concurrente.



- Los proveedores de datos (Opta, StatsBomb) o estándares de la FIFA (EPTS) envían flujos jerárquicos.
- **Granularidad:** Un simple "pase" incluye decenas de metadatos asociados.
- La comprensión de estos esquemas JSON es vital para el diseño de bases de datos vectoriales y algoritmos analíticos.



Campo (Opta/EPTS)	Tipo	Implicación Analítica
event_id	UUID	Deduplicación semántica (exactly-once).
type_id	Entero	Filtrado en capa de ingesta (separar pases de tiros).
start_x, start_y	Flotante	Coordenadas que alimentan modelos espaciales.
timestamp	Time	Reloj del evento, crítico para latencias.
qualifiers	JSON	Contexto profundo (presión, pie débil) para enriquecer prompts de la IA.



- **Paralelismo horizontal:** El número de particiones define el rendimiento máximo.

$$\text{Particiones} = \frac{\text{Throughput total deseado}}{\text{Velocidad máxima del consumidor}}$$

- **Eficiencia:** Afinación de `batch.size` y `linger.ms` combinados con compresión (Snappy/Gzip) para reducir la carga de red.
- **Resiliencia:** Uso de `acks=all` y `enable.idempotence=true` para garantizar que ningún dato crítico (como una alerta de fatiga) se pierda o duplique.



- Se trata al flujo ininterrumpido de eventos como una tabla relacional en crecimiento perpetuo.
- Permite a los científicos de datos abstraerse de la complejidad subyacente.
- Motor de cálculo incremental para preparar métricas (ej. Posesión en el último tercio) antes de enviarlas al modelo de IA generativa.



- En un estadio, los dispositivos se desconectan y reconectan continuamente.
- Spark gestiona la semántica exactly-once usando:
 - **Write-Ahead Logs (WAL)** en sistemas distribuidos persistentes (HDFS/S3).
 - **Checkpointing distribuido:** Guarda el progreso de manera asíncrona.
- Asegura que el LLM no reciba alertas duplicadas sobre el mismo evento de juego.



- **Problema de asincronía:** El retraso entre que el jugador hace el sprint y el paquete de datos llega al servidor en la nube.
- **Watermarking:** Umbral temporal de gracia que el sistema espera antes de cerrar el cálculo de una ventana.

$$W(t) = T_{max}(t) - \Delta$$

- Garantiza que la IA analice ventanas de tiempo completas sin colapsar la memoria del servidor.



- **Desventaja de Lambda:** Mantener código dual (Speed y Batch) duplica el riesgo de error algorítmico.
- **Ventaja de Kappa para la IA:** Kafka persiste todo el historial; el motor de streaming reprocesa todo desde cero si cambia un modelo subyacente.
- Garantiza que el agente cognitivo de IA reciba distribuciones de datos y métricas idénticas, ya sea auditando un partido de 2024 o analizando el directo.

IA Generativa y flujos agénticos



- La interacción ya no es reactiva (humano preguntando a una máquina).
- Se avanza hacia arquitecturas **agénticas y orquestadas orientadas a la acción** (Agentic Workflows).
- Un agente no solo resume texto, sino que ejecuta código, consulta bases de datos vectoriales y planifica respuestas estructuradas.



- LangChain funciona como el marco orquestador.
- Interconecta el razonamiento del LLM con las capacidades deterministas de Big Data (Spark, bases de datos SQL).
- Aporta "herramientas" (Tools) al LLM para que este no invente números, sino que los busque activamente en los clústeres de datos.



Uso práctico: Un analista interroga al sistema en lenguaje natural.

- 1 El LLM usa su herramienta integrada para traducir la consulta humana a un código de consulta (Spark SQL).
- 2 Ejecuta la consulta contra el DataFrame en memoria.
- 3 Extrae el resultado numérico.
- 4 Redacta una explicación en base a ese dato duro.



Pregunta del entrenador: "¿Tasa de éxito de pase progresivo del mediocentro bajo presión en los últimos 15 min?"

- **Planner:** Descompone la consulta → Entidad (jugador), Métrica (tasa de pase), Contexto (presión alta basada en coordenadas), Filtro Temporal (15 min).
- **Executor:** Corre el SQL en Spark.
- **Synthesizer:** Valida lógicamente el 45% obtenido y genera la alerta narrativa argumentada.



- La automatización proactiva que monitorea patrones complejos sin requerir un prompt humano.
- Se utilizan máquinas de estado explícitas para enrutar decisiones.
- Compañías como **Machina Sports** ya proveen infraestructuras de IA agéntica enfocadas exclusivamente en el ecosistema deportivo, dotando a la IA de conocimiento táctico profundo y ejecución fiable.



El sistema opera en un ciclo de iteración continuo:

- ① **Think:** "Detecto una anomalía en la velocidad del jugador."
- ② **Act:** "Usaré la herramienta de historial médico para ver sus promedios."
- ③ **Observe:** "La recuperación cardíaca ha bajado un 15% respecto al mes pasado."
- ④ **Act:** "Emitiré una alerta de sustitución al cuerpo técnico."



Para garantizar estabilidad computacional frente a alucinaciones:

- **Router:** Desvía consultas puramente estadísticas hacia motores deterministas y consultas cualitativas hacia RAG.
- **Validator:** Obliga al agente a parsear su salida bajo un formato estricto (JSON), repitiendo el proceso si el formato es erróneo.
- **Loop Bounded:** Impone un máximo de "pensamientos" iterativos para evitar que el sistema agote los tokens en un bucle infinito de resolución.

RAG Multimodal y Generación de Narrativas



- Los datos tabulares solo representan una fracción de la realidad del partido.
- **RAG Multimodal:** Integrar simbióticamente flujos de vídeo HD, audio de retransmisión y telemetría estructurada.
- **Vectorización combinada:** Permite a la IA no solo buscar "tiros a puerta", sino buscar "jugadas con presión asfixiante" utilizando comprensión espaciotemporal.



- Sistemas de visión profunda analizan fotogramas y detectan la topología del juego y trayectorias.
- **Arquitecturas avanzadas (ej. GameSight):** Realizan un alineamiento en dos etapas, identificando primero las entidades visuales anónimas en el vídeo para luego enriquecerlas con metadatos y estadísticas históricas.
- Esto mejora drásticamente la calidad de los comentarios automatizados.
- Erradica la disonancia de contexto en los modelos de lenguaje genéricos.



- **Problema fundamental:** Inyectar payloads JSON crudos al contexto del LLM es ineficiente y provoca pérdida de precisión matemática (alucinaciones).
- **Solución:** Preprocesamiento determinista.
- Traducir diccionarios numéricos (`type_id = 10`, `outcome = 0`) a oraciones semánticas estructuradas antes del prompt: "El jugador X intentó una entrada pero no recuperó la posesión".

Cada vez es menos necesaria esta conversión, pero no es deseable introducir JSONs muy grandes.



- Creación de pipelines cerrados de monitorización continua de eventos para generar contenido automatizado y filtrado.
- Sistemas como **LLM-SportTweet** utilizan orquestación para leer flujos de eventos asíncronos y tomar decisiones editoriales determinando qué sucesos merecen ser convertidos en narrativas.



Antes de generar una alerta o informe, el LLM evalúa y puntúa internamente el evento táctico o la noticia:

- **Proximity:** Relevancia para el deporte/equipo analizado.
- **Freshness:** Criticidad e inmediatez del suceso.
- **Impact:** Grado de influencia en el resultado del partido.
- **Uniqueness:** Penalización de la redundancia frente a la memoria episódica a corto plazo.



- Solo si el evento supera el umbral de Scoring, el agente desencadena la generación.
- **Técnicas Few-Shot:** Forzar al modelo a adoptar un formato retórico y una longitud específica.
- Se adapta el vocabulario automáticamente:
 - **Modo Analista:** Foco estadístico en métricas de presión.
 - **Modo Aficionado:** Foco emocional y contexto histórico para redes sociales.



- La generación de narrativas en directo exige reducir la latencia al mínimo.
- Se investigan soluciones híbridas que acoplan redes neuronales convolucionales con LLMs.
- **Ejemplo:** Arquitecturas OS-CNN pre-computan y anticipan eventos cinemáticos que luego se inyectan en plantillas Prompt Engineering para modelos como LLaMA 3.3, logrando comentarios fluidos en tiempo real con latencias aptas para retransmisiones interactivas.



La fiabilidad estricta no es opcional en la élite.

Principio de Separación (Separation of Concerns):

- El LLM jamás computa matemáticas (complejas).
- El motor Spark realiza las agregaciones.
- El marco RAG localiza el dato duro.
- El LLM ingiere el dato protegido bajo el mandato estricto de "Utilice EXCLUSIVAMENTE los parámetros de la matriz".

Visualización Avanzada e Interfaces



- Los petabytes de datos son inútiles sin una democratización efectiva de los insights.
- **El paradigma 2026:** Las predicciones y gráficas estáticas previas al partido son obsoletas.
- Se demandan probabilidades recalibradas continuamente en función del flujo de eventos (e.g., probabilidades de micro-sucesos) y total transparencia analítica.



- **Cuerpo Técnico:** Resolución analítica máxima; fusión de cinemática de alta frecuencia y fisiología para prevención biomecánica de lesiones.
- **Front Office:** Proyecciones a largo plazo (impacto económico, modelado de carreras de atletas).
- **Broadcasters:** Baja latencia, inmersión rápida. Respuestas dinámicas a escenarios de tipo "What-If" para los espectadores.



- Sustitución de los paneles pasivos (Tableau, Power BI clásicos) por ecosistemas interactivos.
- La IA funciona como "científico de datos virtual permanente".
- Permite a los analistas realizar consultas declarativas profundas (e.g., "Compara nuestro índice de progresión central antes y después de la sustitución del minuto 60").



- **Problema de los Black-Boxes:** Los cuerpos técnicos no confían en índices predictivos opacos.
- La visualización debe incluir **Virtual Analyst Narratives**.
- El sistema no solo emite una probabilidad del 73%, sino que el agente detalla causalidades (e.g., métricas previas, tendencias posicionales) para justificar el número mostrado en pantalla.



- ① **Recepción:** El agente capta el pico en la gráfica.
- ② **Consulta interna:** Formula SQL y consulta el stream de datos en segundo plano.
- ③ **Recuperación RAG:** Acopla métricas actuales con cortes de vídeo del partido en curso.
- ④ **Generación de la narrativa:** Ofrece una síntesis causal explicativa en lenguaje natural para acompañar al gráfico.



- **Time-to-Insight:** Métrica fundacional en el desarrollo de paneles intra-partido.
- **Problema:** La decodificación del modelo generativo añade un retraso inherente.
- **Solución (Streaming UI):** Reducir el Time To First Token (TTFT) inyectando la respuesta palabra por palabra a la interfaz (vía WebSockets).
- Se mitiga la percepción de lentitud del usuario final de forma drástica.



- Durante la competición táctica, la fluidez del panel de control es primordial.
- Frente al teorema CAP, el diseño del panel favorece la tolerancia a la partición y la disponibilidad.
- Es preferible renderizar tendencias estimadas al instante que bloquear el dashboard a la espera de la consistencia total del evento cerrado.

Conclusiones



- El procesamiento de Big Data moderno consolida el streaming continuo con el análisis cognitivo.
- La orquestación agéntica y los RAG multimodales son las claves para dar contexto espacio-temporal a la inteligencia artificial.
- La arquitectura debe mitigar sistemáticamente las alucinaciones limitando el acceso de los modelos a datos precomputados deterministas.
- El futuro radica en visualizaciones conversacionales y proactivas que empoderen a todos los stakeholders del entorno deportivo en tiempo real.



- El texto, los ejemplos y los esquemas de esta edición abierta se distribuyen bajo CC BY-SA 4.0.
- Las imágenes de procedencia no acreditada de la versión de trabajo se han retirado y sustituido por esquemas conceptuales originales.
- Los logotipos y marcas citados se utilizan únicamente con finalidad identificativa y quedan excluidos de la licencia.
- Las referencias técnicas se conservan mediante citas y enlaces a sus fuentes correspondientes.
- Información de licencia: <https://creativecommons.org/licenses/by-sa/4.0/deed.es>

Big Data Processing II

IA Generativa aplicada al deporte

Tema 6: Tendencias y Futuro

Procesamiento Masivo de Datos e IA en el Deporte

Alberto Fernández Isabel
Natalia Madrueño Sierro
Rubén Rodríguez Fernández

DSLAB

Abril 2026





Material docente en abierto de la Universidad Rey Juan Carlos

Asignatura: Big Data Processing II

Titulación: Máster Universitario en Análisis de Datos Deportivos

Autores: Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández

Fecha: 2026

© 2026 Equipo docente. Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional.

Los logotipos, marcas y referencias de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

Depósito institucional: <https://burjcdigital.urjc.es>



- Comprender la transición de la analítica retrospectiva a la prescriptiva en el deporte.
- Explorar tecnologías emergentes actuales: Edge Computing, 5G, Visión Artificial.
- Analizar implicaciones estratégicas y de negocio de la analítica prescriptiva.
- Evaluar tecnologías del futuro próximo: computación cuántica, blockchain, BCI, datos sintéticos.
- Examinar los retos éticos, legales y de gobernanza de datos biométricos.



- 1 **Introducción: De la observación retrospectiva a la prescripción estratégica**
- 2 **Convergencia de infraestructura: El estadio de latencia cero**
- 3 **Visión artificial y desmaterialización de la telemetría**
- 4 **Analítica prescriptiva, orquestación multiagente y automatización**
- 5 **Tecnologías emergentes: Más allá del horizonte actual**
- 6 **Retos éticos, legales y gobernanza de datos**

Introducción: De la observación retrospectiva a la prescripción estratégica



- **Pasado:** Análisis descriptivo post-partido (batch processing).
- **Presente:** Procesamiento en tiempo real con Kafka y Spark Streaming.
- **Futuro:** Analítica prescriptiva que recomienda acciones óptimas automáticamente.
- Transición de "¿qué pasó?" → "¿por qué pasó?" → "¿qué debemos hacer?"
- **Desafío:** El valor del dato decae exponencialmente desde el momento del evento.
- **Objetivo:** Insights accionables durante la competición, no después.



Predictiva

- Anticipa probabilidades
- "¿Cuál es el riesgo de lesión?"
- Requiere interpretación humana

Prescriptiva

- Recomendando acciones
- "Sustituye al jugador X por Y en el minuto 65"
- Evalúa escenarios concurrentes
- Maximiza probabilidad de éxito



- Valoración proyectada: **3.400 millones de dólares para 2028.**
- Impulsores clave:
 - Reducción drástica de costes de computación para IA
 - Comercialización de derechos NIL (Nombre, Imagen, Semejanza)
 - Obsolescencia de cookies de terceros
 - Necesidad de captura de datos propios
- La tecnología pasa de soporte a **núcleo estratégico del negocio.**



- **Juegos Olímpicos de Invierno Milán 2026**
- **Copa Mundial FIFA 2026**
- Actúan como bancos de pruebas de tecnologías disruptivas
- Demuestran que eficiencia, personalización, arbitraje y equidad dependen de infraestructuras convergentes

Convergencia de infraestructura: El estadio de latencia cero



- Excelente para análisis retrospectivo
- Transmitir telemetría de 60.000 espectadores + biosensores de jugadores a centros remotos introduce latencias insalvables
- Retardo incompatible con decisiones intra-partido

Solución: Multi-access Edge Computing (MEC) + Redes 5G Privadas (P5G)



MEC (Edge Computing)

- Cómputo en el estadio
- Latencia: cientos ms → 1 dígito
- Análisis antes de que concluya jugada
- CV opera localmente

Redes 5G Privadas

- Carril dedicado
- Network Slicing
- Evita congestión Wi-Fi
- Ancho de banda garantizado



Protocolos críticos

- **mMTC:** Miles de IoT concurrentes
- **URLLC:** Latencia ultra-baja, alertas médicas

Resultado: Procesamiento distribuido con latencia mínima y ancho de banda optimizado.

Arquitectura federada

- Nodos en estadio
- Agregación continua
- Filtrado local
- Nube: histórico + training



- Redes locales + procesamiento en el borde
- Zonas VIP con gafas de realidad aumentada
- Superposición de estadísticas y mapas de calor en vivo
- Sincronización con acción física sin retraso perceptible

Visión artificial y desmaterialización de la telemetría



Wearables tradicionales

- GPS, acelerómetros
- Alta precisión
- Fricciones:
 - Calibración
 - Baterías
 - Incomodidad
 - Privacidad

Computer Vision (2026)

- Tendencia dominante
- Costes IA en caída
- CNN + pose esquelética
- Sin fricción física
- Escala masiva



- Identificar jugadores inequívocamente (pese a oclusiones)
- Rastrear coordenadas espaciales precisas
- Medir distancias entre líneas defensivas
- Cuantificar índice de intensidad de presión táctica (PPDA)
- Todo de forma completamente automatizada

Mercado: Crecimiento anual del 29.8%, de 2.390 M\$ a 8.700 M\$ para fin de década.



- **Copa Mundial FIFA 2026:** Colaboración FIFA-Lenovo
- Generación de modelos 3D de alta precisión mediante escaneos digitales
- No son estéticos: son la **Ground Truth** del procesamiento
- Interactúan con sistema de fuera de juego semiautomático
- Mejoran precisión al calcular momento de impacto y posición volumétrica
- Se inyectan en retransmisiones para explicar decisiones VAR



- Reducción de desenfoque por movimiento (motion blur)
- Procesamiento en tiempo real de cámaras corporales de árbitros
- Ángulos complicados mejorados algorítmicamente
- Transmisiones fluidas para revisión de jugadas

Visión artificial: de herramienta experimental a **infraestructura central**.

Analítica prescriptiva, orquestación multiagente y automatización



- Calcular trayectorias y métricas ya no es ventaja: es el **estándar mínimo**
- **Valor transformador:** Transición a ecosistemas prescriptivos
 - Descriptivo: "¿Qué pasó en el partido?"
 - Diagnóstico: "¿Por qué perdimos el control?"
 - **Prescriptivo: "¿Qué ajustes debemos implementar?"**



Propuesta de valor

- Reducción lesiones: 15-25%
- Optimización fichajes: 30-40%
- Inversión: 1-3M€
- Break-even: 18-36 meses

Factores críticos

- ✓ Patrocinio ejecutivo
- ✓ Formación staff
- ✓ Pilotos graduales
- × Resistencia cultural
- × Silos departamentales
- × Falta de talento



- **Hipótesis:** ¿La analítica prescriptiva es ventaja sostenible o commodity temporal?
- **Evidencia empírica:**
 - Liverpool FC (2018-2020): Inversión agresiva en ciencia de datos correlaciona con títulos
 - Democratización acelerada (Football AI Pro, soluciones SaaS) reduce brecha tecnológica
- **Conclusión:** Ventaja no reside en *tener* tecnología, sino en:
 - Calidad y granularidad de datos propios
 - Capacidad de integración organizacional
 - Velocidad de adaptación táctica en función de insights



Tendencia 2026: Migración de prototipos a producción (Tema 5: implementación técnica)

- Análisis táctico, búsqueda semántica, informes automatizados
- 40h análisis manual → 4h supervisión

Decisión estratégica:

Build

- Control total
- 2-5M€/año

Buy (SaaS)

- Rápido
- 200-500K€/año

Partner

- Co-desarrollo
- Riesgo compartido



- Los cuerpos técnicos rechazan "Cajas Negras"
- Un sistema efectivo no solo emite: "Cambie a defensa zonal"
- Debe acompañar evidencia trazable:
"Se recomienda cobertura zonal porque la fatiga de su lateral derecho está en percentil 90, y el rival capitaliza contraataques por esa banda en segundas partes."



Football AI Pro (FIFA 2026)

- FIFA + Lenovo
- IA generativa multilingüe
- Disponible 48 selecciones
- Democratiza analítica táctica

La brecha tecnológica se reduce: ventaja sostenible = calidad de datos + integración organizacional.

Nuevos perfiles

- LALIGA Business School
- Máster Transformación Digital
- Programa Ejecutivo IA Deporte
- Analistas Estrategia Algorítmica

Tecnologías emergentes: Más allá del horizonte actual



- **Estado actual:** Investigación experimental, no producción
- **Potencial transformador:**
 - Optimización combinatoria de alineaciones ($11! = 40\text{M}$ permutaciones)
 - Simulación de escenarios tácticos en paralelo cuántico
 - Resolución de problemas NP-hard en planificación de calendarios
- **Casos de uso explorados:**
 - IBM Quantum + Fórmula 1: Optimización de estrategia de neumáticos
 - Algoritmos variacionales (VQE) para análisis de grafos tácticos
- **Barrera:** Hardware cuántico aún limitado (qubits < 1000 , alta tasa de error)
- **Horizonte:** 2030-2035 para aplicaciones prácticas en deporte de élite



- **Problema actual:** Centralización de datos biométricos en proveedores tecnológicos
- **Propuesta blockchain:**
 - Atletas retienen propiedad criptográfica de sus datos
 - Smart contracts definen quién accede, cuándo y bajo qué condiciones
 - Registro inmutable de consentimientos y usos
- **Casos de uso en desarrollo:**
 - NFTs de firmas cinemáticas únicas (monetización NIL)
 - Marketplace descentralizado para licencias de datos
 - Auditoría transparente de cumplimiento GDPR/BIPA
- **Retos:** Escalabilidad (throughput), consumo energético, complejidad técnica



- **Más allá de wearables:** Captura directa de señales neuronales
- **Aplicaciones prospectivas:**
 - Medición de carga cognitiva en tiempo real (toma de decisiones bajo presión)
 - Detección temprana de fatiga central (antes de manifestación física)
 - Optimización de estados de flow en entrenamiento
- **Tecnologías actuales:**
 - EEG no invasivo: precisión limitada, ruido de movimiento
 - Implantes neuronales (Neuralink): invasivos, dilemas éticos
- **Horizonte:** 2028-2030 para BCI no invasivos portátiles
- **Barrera crítica:** Aceptación ética y regulación de neurotecnología



- **Problema:** Escasez de datos etiquetados + restricciones de privacidad (GDPR, BIPA)
- **Solución:** Generar datos sintéticos estadísticamente equivalentes
 - GANs (Redes Generativas Adversarias) entrenadas en datos reales
 - Producen millones de trayectorias sintéticas de jugadores
 - Preservan distribuciones estadísticas pero no identidades reales
- **Ventajas:**
 - Entrenamiento de modelos ML sin exponer datos sensibles
 - Compartir datasets entre clubes sin violar contratos
 - Aumentar diversidad de casos extremos (lesiones raras)
- **Desafío:** Garantizar que datos sintéticos no permiten re-identificación

Retos éticos, legales y gobernanza de datos



- Capturar millones de puntos biométricos, cinemáticos y tácticos confiere ventaja asimétrica
- Telemetría íntima introduce fricciones normativas sin precedentes
- Dilemas morales profundos
- **Gobernanza ética:** Pilar estructural, no requisito accesorio
- Evita que optimización degenera en vigilancia laboral encubierta



RGPD (Reglamento General de Protección de Datos):

- Dato biométrico = categoría especial, protección reforzada
- Consentimiento libre, explícito e informado obligatorio
- **Privacy-by-Design:** Privacidad desde el diseño
- **DPIA:** Evaluaciones de Impacto previas al despliegue
- Derechos de acceso, rectificación y supresión



- **BIPA (Illinois):** Uno de los estándares más estrictos
 - Políticas escritas y publicadas sobre retención
 - Destrucción obligatoria tras cumplir finalidad
 - Límite máximo: 3 años desde última interacción
- **TRAIGA (Texas) - Vigencia enero 2026:**
 - Penaliza captura ilícita de biometría
 - Prohíbe discriminación automatizada
 - Prohíbe social scoring
- **Ley de IA de Colorado:**
 - Evaluaciones de impacto periódicas
 - "Cuidado razonable" de implementaciones algorítmicas



DPA con proveedores

- Acuerdos restrictivos
- Evitar extracción subrepticia
- No usar datos para entrenar modelos sin consentimiento

Relaciones laborales

- Atleta = productor originario
- Sindicatos (NFL, MLB):
 - Bloquean uso punitivo
 - Prohíben negociación salarial
 - Consentimiento expreso
 - Derechos auditoría



- **NIL (Nombre, Imagen y Semejanza):** Expansión masiva
- Datos biomecánicos únicos = activos digitales
- Avatares 3D, firmas cinemáticas exclusivas
- **Conflicto:** Ligas empaquetan métricas para apuestas/VR sin compensar al jugador



Sesgos algorítmicos

- Datos históricos con deficiencias humanas
- Infrarrepresentación:
 - Solo ligas masculinas europeas
 - Inferencias sesgadas para otros grupos
- **Resultado:** Discriminación

Problema Caja Negra

- Sin explicación de variables
- Socava transparencia
- Etiquetado prematuro:
 - "Bajo potencial"
 - "Frágil"
 - Estigmas de mercado



- **Human-in-the-Loop (HITL):**
- IA opera como apoyo complementario
- No reemplaza evaluación integral humana
- Resultados prescriptivos requieren validación
- Combina:
 - Precisión algorítmica
 - Intuición y experiencia del especialista
 - Contexto situacional

Conclusiones



- La convergencia de Edge Computing, 5G y visión artificial redefine el ecosistema deportivo actual.
- La analítica prescriptiva es decisión estratégica de negocio, no solo técnica: ROI, madurez organizacional y ventaja competitiva determinan el éxito.
- Tecnologías emergentes (quantum, blockchain, BCI, datos sintéticos) marcarán la próxima década: preparación anticipada es clave.
- La democratización tecnológica (Football AI Pro) reduce brechas, trasladando ventaja a calidad de datos e integración organizacional.
- La gobernanza ética de datos biométricos no es opcional: es fundacional para la sostenibilidad del sistema.
- El futuro exige profesionales híbridos: dominio técnico + comprensión estratégica + ética aplicada.



- El texto, los ejemplos y los esquemas de esta edición abierta se distribuyen bajo CC BY-SA 4.0.
- Las imágenes de procedencia no acreditada de la versión de trabajo se han retirado y sustituido por esquemas conceptuales originales.
- Los logotipos y marcas citados se utilizan únicamente con finalidad identificativa y quedan excluidos de la licencia.
- Las referencias técnicas se conservan mediante citas y enlaces a sus fuentes correspondientes.
- Información de licencia: <https://creativecommons.org/licenses/by-sa/4.0/deed.es>